

Retro 68K Emulator

(User Guide)

Version 1.4.3

Generated: 2026-09-26

Table of Contents

Presentation

Downloads

References and Guidelines

Troubleshooting

Presentation

The Retro 68K Emulator is a free desktop app for learning Motorola 68000 assembly language, in the style of a retro fantasy console. Write 68000 assembly, run it, and watch the CPU registers, flags, and a simulated 320×200 LCD screen react in real time — no physical hardware required.

It's aimed at students, hobbyists, and anyone curious about how a CPU actually executes instructions, one step at a time.

If you just want to try the app, grab the latest version from the [Downloads](#) page, then load one of the fifteen built-in programs from the editor's "Load an example" menu and press Run.

Features

- Every non-privileged 68000 instruction, plus a built-in assembler
- Full register set: `D0 – D7` (data), `A0 – A7` (address, with `A7` as the stack pointer)
- Status flags: Negative, Zero, Overflow, Carry, Extend
- A TRAP-based system-call layer, for things like printing text or drawing to the screen from your own assembly code
- A simulated 320×200 color LCD (32-bit RGBA), mapped directly into memory
- An on-screen retro gamepad (A/B/X/Y, D-pad, Start/Select), superseded by a real gamepad when one is connected
- Keyboard input: visible ASCII and accented (Latin-1) characters, read a queued character at a time from your own assembly code
- Simple retro-style sound effects (a tone generator, not sampled audio), the way period 8/16-bit consoles did it
- A step-by-step debugger: watch registers, flags, memory (hex dump) and the screen change instruction by instruction, with breakpoints

- A syntax-highlighted assembly editor, built into the app — no external tools needed: Tab/Shift+Tab indent and outdent to 8-column stops, and Enter carries the current line's indentation onto the next one
- Bookmarks: mark lines (right-click a line number, or Ctrl+B), jump between them with F2 / Shift+F2; bookmarks and breakpoints are remembered for each file opened from disk
- Find in code: Ctrl+F opens a find bar, F3 / Shift+F3 cycle through matches
- Copy/Cut/Paste, with no selection acting on the whole current line, the same way most code editors do
- Native installers for Windows, macOS, and Linux, so it runs like any other desktop app

Why This Project?

Most ways to learn assembly language either require real, aging hardware or a fairly unfriendly command-line toolchain. This project's goal is a self-contained, visual, approachable way to write and run 68000 assembly, see exactly what each instruction does to the CPU, and build real intuition for how a computer executes code at the lowest level.

Next: [Downloads](#) · [References and Guidelines](#) · [Troubleshooting](#)

Downloads

At least one version is available. The latest one is on the Releases page linked below.

How releases work

This project builds real, native installers for Windows, macOS, and Linux — no need for Node.js, Rust, or a terminal to run it once it's installed. Each version is published on the project's GitHub Releases page:

<https://github.com/loloof64/retro-68k-emulator/releases>

Each release includes one installer per platform:

Platform	File to download	How to install
Windows	<code>.msi</code> or <code>.exe</code>	Double-click, follow the installer
Windows (portable)	<code>-windows-portable.zip</code>	Unzip anywhere and double-click <code>Retro68K-Emulator.exe</code> — nothing gets installed
macOS	<code>.dmg</code>	Open it, drag the app to <code>Applications</code>
Linux	<code>.deb</code> , <code>.AppImage</code> , or <code>.rpm</code>	<code>.deb</code> / <code>.rpm</code> : install via your package manager. <code>.AppImage</code> : make it executable and run it directly

The Windows `.msi` comes in three languages (installer language only, not the app's — the app itself is multilingual and its language can be changed at any time): `en-US` (English), `fr-FR` (French) and `es-ES` (Spanish) — pick the one you prefer.

On Windows and Linux you don't need to pick an architecture: pick the file matching your operating system. On macOS there are two `.dmg` files: the `aarch64` one for Apple Silicon (M1 and later), the `x64` one for Intel Macs.

The portable Windows zip contains the program and everything it needs to start; it relies only on Microsoft's WebView2 component, which comes with Windows 10 and 11.

Supported systems

System	Supported versions	Notes
Windows	Windows 11 (Windows 10 should also work)	Needs Microsoft's WebView2, already included in Windows 10/11
macOS	macOS 10.15 (Catalina) or later	Intel and Apple Silicon builds — not tested yet
Linux	Ubuntu 22.04 or later, or any distribution with a comparable system (glibc 2.35 or later, WebKitGTK 4.1)	Linux builds are made on Ubuntu 22.04

On Linux, the `.AppImage` and `.deb` need at least glibc 2.35 and WebKitGTK 4.1 (the versions shipped with Ubuntu 22.04 and Debian 12); older distributions will refuse to start the app or to install it. Only Windows 11 and Linux are confirmed by the author. The macOS builds have not been tested at all: they are produced by the same automated build, but nobody has run them yet.

Staying up to date

Watch the GitHub repository (or check the Releases page directly) to know when a new version becomes available.

References and Guidelines

This page documents what you can actually *do* today. Everything is covered — the CPU, the assembler, the memory map, the TRAP system calls (including [sound](#)) and the debugger. See [Presentation](#) for the full roadmap.

Registers

The CPU core (registers, status flags, and the fetch-decode-execute loop) is implemented and working.

Register	Size	Purpose
D0 – D7	32-bit	Data registers — general-purpose, for values and arithmetic
A0 – A7	32-bit	Address registers — hold memory addresses. A7 doubles as the Stack Pointer (SP)
PC	32-bit	Program Counter — address of the next instruction to fetch

D0 – D7 and A0 – A7 are ordinary registers: any instruction that takes a register operand can freely read or write them (`MOVE D1,D0` , `ADD.L D1,A2` , and so on). PC cannot be read or written directly — there's no instruction that `MOVE` s a value into or out of it — it only changes as a side effect of control-flow instructions (`JMP` , `JSR` , `Bcc` , `RTS` , ...).

A freshly created CPU — internally, `createCPU()` / `reset()` in the emulator's core — starts with every register at 0 except A7 , which starts at `$03FFF` (the top of the default stack, which grows downward). The Debugger panel's "🔄 Reset" button does exactly this, then reloads the assembled program and points the CPU at its entry address.

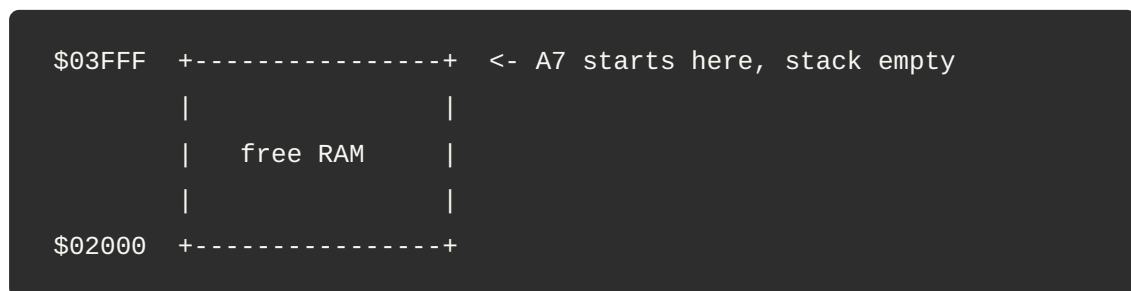
The Stack

The stack is a region of memory used as a last-in-first-out (LIFO) buffer: the last value pushed onto it is the first one popped back off, like a stack of plates you can only add to or take from at the top. The CPU uses it for return addresses (`JSR` / `RTS`), for saving registers around a subroutine call, and for anything a program wants to stash temporarily.

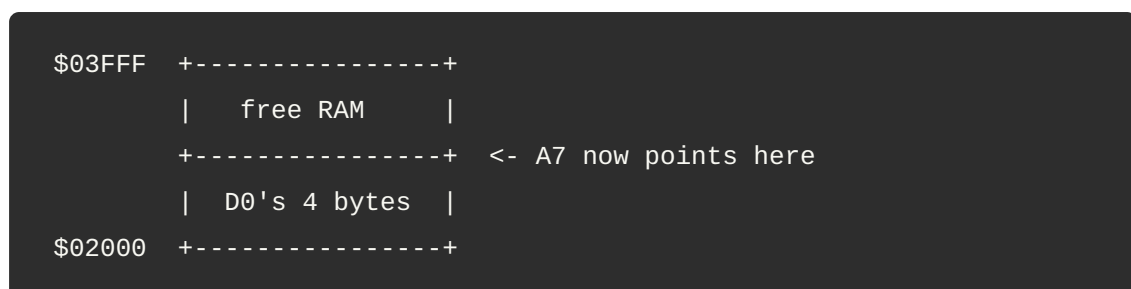
There's no dedicated push/pop instruction — "push" and "pop" are just `MOVE` (or `MOVEM`, `PEA`, `JSR`) targeting `A7` through two addressing modes (see [Addressing Modes](#)):

- **Push** = `-(A7)` : decrement `A7` by the operand's size, *then* write the value there (`MOVE.L D0, -(A7)`).
- **Pop** = `(A7)+` : read the value at `A7`, *then* increment `A7` by the operand's size (`MOVE.L (A7)+, D0`).

Because a push decrements first, **the stack grows downward**, from high addresses toward low ones — the opposite of how User RAM is normally read top-to-bottom. That's why a freshly reset CPU starts `A7` at `$03FFF`, the *top* of the default stack (see [Registers](#) above): nothing has been pushed yet, and each push eats into User RAM from that high end down.



After `MOVE.L D0, -(A7)` pushes a 4-byte value:



`A7` — the Stack Pointer — is what makes this "the" stack rather than just memory: every pushing/popping instruction always targets it specifically,

never another address register. `JSR / BSR` push a return address and `RTS` pops it back off (see [What is a return address?](#)); `PEA` pushes a computed address; `MOVEM list, -(A7)` / `MOVEM (A7)+, list` push or pop several registers in one instruction (see [How does MOVEM's register list work?](#)); `LINK / UNLK` build and tear down a whole stack frame (see [below](#)).

Status Flags

The status flags aren't something a program sets directly — they're a side effect: each instruction that "affects flags" updates them to reflect the result it just produced (a subtraction that hit zero sets `Z`, one that went negative sets `N`, and so on), and they simply hold that value until the next instruction that touches them runs.

Flag	Name	Meaning
<code>N</code>	Negative	Set when the result's sign bit is <code>1</code>
<code>Z</code>	Zero	Set when the result is <code>0</code>
<code>V</code>	Overflow	Set on signed overflow
<code>C</code>	Carry	Set on unsigned carry/borrow
<code>X</code>	Extend	Mirrors <code>C</code> for most operations; used in multi-precision arithmetic

Which flags a given instruction touches is listed per-instruction below, in the "Flags affected" column, using three notations: a flag on its own (e.g. `N`, `Z`) is set or cleared to reflect what the instruction actually produced; a flag followed by `(0)` (e.g. `V (0)`) is unconditionally cleared to `0`, regardless of the result — real hardware does this where the flag has no meaningful value for that instruction (multiply/divide can't overflow the way add/sub can, so `MULU / MULS` always clear `V`); and a flag missing from the list entirely is left untouched, keeping whatever value it had before the instruction ran. So "none" (or a missing flag) does **not** mean the flags are reset to `0`: they simply keep their previous values (for example,

`DBRA` never touches them, so a flag set by the previous instruction stays lit through the whole loop).

What are SR and CCR?

The five flags live together in one register, the **Status Register**, written `SR`. It is 16 bits wide: the flags occupy its low 5 bits (`X` is bit 4, `N` bit 3, `Z` bit 2, `V` bit 1, `C` bit 0), and the high byte holds things this emulator doesn't model (supervisor mode, interrupt mask, trace). The **Condition Code Register**, written `CCR`, is simply the low byte of `SR`: the flags and nothing else. That is why some instructions target `CCR` (`MOVE src,CCR`, `ANDI #data,CCR`) to set or clear flags directly, while `MOVE SR,dst` reads them back out into a register or memory. You never need either to write ordinary programs: arithmetic and comparisons set the flags for you, and branches read them.

What does cc mean?

In `Bcc`, `DBcc` and `ScC` op-codes, `cc` is a placeholder for a *condition*: a two-letter test on the flags, such as `EQ` (equal, `Z=1`), `NE` (not equal), `GT` (greater than, signed) or `CS` (carry set). Replace `cc` with one to get an actual instruction: `BEQ`, `DBNE`, `SGT`. The full list, and which ones to use after a `CMP`, is in [Which Bcc do I want?](#). (Motorola's manuals also call the flags themselves "condition codes", hence the name `CCR`; here, `cc` only ever means one of these tests.)

Instruction Format

Every instruction in the tables below is written using the same template:

```
MNEMONIC.SIZE src,dst
```

- **MNEMONIC** is the instruction's name — `MOVE`, `ADD`, `BRA`, and so on. It's what tells the CPU *what* to do.

- **.SIZE** is an optional suffix controlling how many bits of data the instruction touches: `.B` for a byte (8 bits), `.W` for a word (16 bits), or `.L` for a long word (32 bits). Not every instruction supports every size — the

"Sizes" column in each instruction table below lists which ones a given mnemonic accepts. Leaving the suffix off entirely defaults to `.w` (word) for instructions that support it. The assembler rejects a suffix the instruction cannot take (`NOP.L` , `MULU.L`) rather than ignoring it, and `.s` (a short branch) exists only on `BRA` , `BSR` and `Bcc` . When `dst` is a data register (`Dn`), `.b` / `.w` only overwrite the low byte/word — the rest of the register keeps whatever it held before, it isn't cleared. An address register (`An`) destination works differently: there's no partial write, a `.w` source is sign-extended to fill the full 32 bits instead (see `MOVEA` in [Instruction Set \(Opcodes\)](#)). **Sign-extended** means the value's top bit (its sign: 0 for positive, 1 for negative) is copied into every new upper bit being added, instead of those bits just being set to 0 — that's what makes a negative 16-bit value like `$FFFF` (-1) turn into the 32-bit `$FFFFFFFF` (still -1) rather than `$0000FFFF` (a large positive number instead).

- **src** and **dst** are the operands — "source" (where a value comes from) and "destination" (where it goes). Each one is written using one of the addressing modes described just below, e.g. a register (`D0`), a memory location (`(A0)`), or a constant (`#100`).

Not every instruction takes both a source and a destination — some take only one operand (e.g. `NOT.L D0` , which flips every bit of `D0` in place), and some take none at all (e.g. `RTS` , which just returns from a subroutine). The exact operands each instruction accepts are spelled out in its own "Syntax" column further down this page, in [Instruction Set \(Opcodes\)](#) — the template above is just the general shape they all follow.

The 68000 is **big-endian**: for a `.w` or `.l` value, the most significant byte is stored at the lowest memory address. Writing `$1234` as a word therefore leaves the bytes `$12` then `$34` , in that order, at increasing addresses — see [Inspecting Memory](#) for a worked example.

Addressing Modes

An addressing mode is how an instruction says where an operand lives — a register, a memory address, or a constant baked right into the instruction. Elsewhere on this page, `<ea>` ("effective address") stands for "any operand written using one of the addressing modes below" — it

shows up in syntax like `ADD <ea>, Dn` to mean the source can be *any* of them (a register, `(A0)`, `#5`, ...), as opposed to a fixed operand like `Dn`, which only ever means a data register. Not every instruction accepts every mode as its `<ea>` — restrictions (e.g. no `#imm` as a destination) are called out per instruction in [Instruction Set \(Opcodes\)](#) and [Instruction Usage Guidance](#). These are the modes wired in today:

Mode	Syntax	Example	Description
Data register	<code>Dn</code>	<code>MOVE.L D0, D1</code>	The value in a data register
Address register	<code>An</code>	<code>MOVE.L A0, A1</code>	The value in an address register
Register indirect	<code>(An)</code>	<code>MOVE.L (A0), D0</code>	The value in memory at the address held in <code>An</code>
Post-increment	<code>(An)+</code>	<code>MOVE.L (A0)+, D0</code>	<code>An</code> is used as the address first, <i>then</i> bumped by the operand's size
Pre-decrement	<code>-(An)</code>	<code>MOVE.L D0, -(A0)</code>	<code>An</code> is decremented by the operand's size first, then used as the address
Displacement	<code>d16(An)</code>	<code>MOVE.L \$10(A0), D0</code>	<code>An</code> plus a 16-bit displacement — <code>An</code> itself is left unchanged
Immediate	<code>#value</code>	<code>MOVE.L #100, D0</code>	A constant baked into the instruction (source only — can't be a destination)

Mode	Syntax	Example	Description
Absolute short	<code>xxx.W</code>	<code>MOVE.L \$100.W, D0</code>	A 16-bit address, sign-extended — can only point at <code>\$0000 - \$7FFF</code> : the whole System area (<code>\$0000 - \$1FFF</code>) plus the first 24 KB of User RAM, not far enough to reach the Framebuffer, Controller Input, or Sound (or, on real hardware, the top of memory too; this emulator's address space doesn't extend that far)
Absolute long	<code>xxx.L</code>	<code>MOVE.L \$40000.L, D0</code>	A full 32-bit address, written directly into the instruction — can point at any region : System area, User RAM, Framebuffer, Controller Input, or Sound

Mode	Syntax	Example	Description
Indexed	<code>d8(An, Xn)</code>	<pre>MOVE.L \$10(A0, D1.W), D0</pre>	<code>An</code> plus an index register, <code>Xn</code> (<code>Dn</code> or <code>An</code>, <code>.W</code> sign-extended or <code>.L</code>) plus an 8-bit displacement — like Displacement above, both <code>An</code> and <code>Xn</code> are only read, neither is changed
PC displacement	<code>d16(PC)</code>	<pre>MOVE.L \$10(PC), D0</pre>	The program counter (at the displacement's own extension word) plus a 16-bit displacement — source only, can't be a destination; like Displacement above, <code>PC</code> is only read, not changed by this addressing mode itself (it still advances normally past the extension word, same as fetching any other instruction)

Mode	Syntax	Example	Description
PC indexed	<code>d8(PC, Xn)</code>	<code>MOVE.L \$10(PC, D1.W), D0</code>	Like Indexed, but based on the program counter instead of an address register — source only; neither <code>PC</code> nor <code>Xn</code> is changed by this addressing mode itself

Absolute addressing works either as shown above or, for an address you'll reuse, by loading it into an address register first with `MOVEA` (e.g. `MOVEA.L #$40000, A0` then `(A0)`) — most examples on this page still use the `MOVEA` style since it's what the addressing modes actually looked like before absolute addressing landed, but either works today.

Displacement (`d16(An)`) only ever *reads* `An` to compute the address — unlike post-increment and pre-decrement, it never writes a new value back into it. Use it when you want to reach an offset from `An` without moving `An` itself (e.g. `MOVE.L 4(A0), D0` to read the second long word of a struct pointed to by `A0`, leaving `A0` where it was for the next access); use post-increment/pre-decrement instead when you're walking through memory one element at a time and want `An` to advance (or retreat) as you go.

Memory Map

The emulator's memory system is implemented and working. It's entirely simulated inside the app — a plain in-memory buffer, never your computer's actual RAM:

Region	Address range	Size	Purpose
System area	<code>\$000000</code> – <code>\$01FFFF</code>	8 KB	Reserved for TRAP vectors and system data

Region	Address range	Size	Purpose
User RAM	\$02000 – \$3FFFF	~248 KB	Your program's code, data, and stack
Framebuffer	\$40000 – \$7E7FF	250 KB	The 320×200 screen, 4 bytes per pixel, as a 32-bit \$RRGGBBAA value (red \$FF0000FF)
Controller Input	\$7E800 – \$7E803	4 B	Gamepad button state, as a bitmask (see below)
Sound	\$7E804 – \$7E80B	8 B	Tone generator registers, written by TRAP #6 — played through Web Audio (see below)

To find the address of a given pixel, see [Addressing a Pixel for TRAP #2/#3](#).

Colors

A color is one 32-bit value, written \$RRGGBBAA in hexadecimal: two hex digits (one byte, \$00 – \$FF) each for red, green, blue and alpha. The same format is used everywhere a color appears: pixels in the framebuffer, the D2 color of TRAP #1 , and the D0 value of TRAP #2 , #3 and #4 .

```

31      24 23      16 15      8 7      0
+-----+-----+-----+-----+
|  RED   |  GREEN  |  BLUE  |  ALPHA  |
| $00-$FF | $00-$FF | $00-$FF | $00-$FF |
+-----+-----+-----+-----+
```

Each component runs from \$00 (none) to \$FF (full). The 68000 stores the most significant byte first, so in memory a pixel's four bytes are red, green, blue, alpha, in that order, at increasing addresses.

Alpha is ignored: the screen has no transparency, so \$FF0000FF and \$FF000000 both show pure red. By convention programs write \$FF there (*opaque*). This is also why a pixel of \$00000000 shows as black.

Color	Value
Black	\$000000FF
White	\$FFFFFFF
Red	\$FF0000FF
Green	\$00FF00FF
Blue	\$0000FFFF
Yellow	\$FFFF00FF
Cyan	\$00FFFFFF
Magenta	\$FF00FFFF
Orange	\$FF8000FF

To read one component back out of a color, *shift* it down to the lowest byte, then *mask* off what is above it. A shift moves every bit of a register by a given number of places (`LSR` , "logical shift right", moves bits toward the low end and fills created "holes" with zeros); a mask is an `AND` with `$FF` , which keeps only the low 8 bits and clears the rest. The red byte is already the top one, so shifting by 24 leaves only it:

```

MOVE.L  #$FF8040FF,D0    ; a color: R=$FF G=$80 B=$40 A=$FF
MOVE.L  D0,D1
MOVEQ   #16,D2            ; shift count: 16 bits
LSR.L   D2,D1             ; D1 = $0000FF80
ANDI.L  #$FF,D1          ; D1 = $00000080: the green byte
MOVE.L  D0,D3
MOVEQ   #24,D2
LSR.L   D2,D3             ; D3 = $000000FF: the red byte
MOVE.L  D0,D4
ANDI.L  #$FF,D4          ; D4 = $000000FF: the alpha byte

```

The blue byte needs a shift by 8 and the same mask. To go the other way and *build* a color from components, shift left (`LSL`) and `OR` the pieces

together, or simply write the whole `$RRGGBBAA` value as one immediate, which is what the built-in example programs do.

Reading the Gamepad

`$7E800` is a live 32-bit bitmask of the current button state — bit `1` means held down. The app's Gamepad component keeps it updated from whichever input is active: the on-screen A/B/X/Y + D-pad + Start/Select control pad by default, or a real controller's buttons the moment one is connected (a standard-mapped gamepad takes over automatically — the on-screen buttons stop doing anything while it's plugged in).

The four face buttons follow their **position**, like on an Xbox controller: A is the bottom button, B the right one, X the left one and Y the top one. On a Nintendo-style controller, whose A/B and X/Y are laid out the other way round, the button printed "A" (on the right) therefore lights B, and the button printed "X" (at the top) lights Y — the physical position is what stays the same from one controller to another.

Bit	Button
0	A
1	B
2	X
3	Y
4	D-Pad Up
5	D-Pad Down
6	D-Pad Left
7	D-Pad Right
8	Start
9	Select

```

TRAP    #5                ; D0 = controller state
BTST    #0,D0             ; test bit 0 (button A)
BEQ     A_NOT_PRESSED     ; Z=1 -> button A isn't held

```

Sound

`$7E804` – `$7E80B` is laid out for a single-voice tone generator — frequency, duration, volume, waveform — the same idea as a PC speaker or the TI-89's buzzer, not a sample player.

Offset	Field	Size	Purpose
+0	Frequency	word	Hz; 0 = silence
+2	Duration	word	milliseconds
+4	Volume	byte	0 - 255
+5	Waveform	byte	0 =square 1 =sine 2 =triangle 3 =sawtooth 4 =noise
+6	Trigger	byte	nonzero after TRAP #6

TRAP #6 writes D0-D3 into those fields in one step and sets the trigger:

```

MOVE.W  #440,D0           ; frequency (Hz)
MOVE.W  #250,D1           ; duration (ms)
MOVE.B  #200,D2           ; volume
MOVE.B  #0,D3             ; waveform (0 = square)
TRAP    #6

```

The debugger watches the trigger after every instruction: it plays the tone through the Web Audio API and clears the trigger back to 0. Only one tone plays at a time — a new one cuts the previous. A frequency or duration of 0 is silence. Sound starts after you press **Run** or **Step** (browsers refuse audio before a click).

Instruction Set (Opcodes)

Every non-privileged 68000 instruction is listed below, grouped the way Motorola's own 68000 Programmer's Reference Manual groups them. **Cycles** are how many CPU cycles an instruction takes to run — smaller is faster; they're what the emulator's cycle counter adds up as your program executes.

(Real 68000 hardware charges different cycle counts per addressing mode, and `Bcc` costs less when the branch isn't taken — this emulator deliberately doesn't model that: each instruction below just charges one flat cycle count, regardless of addressing mode or whether a branch was taken.)

Data Movement

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
<code>MOVE</code>	<code>MOVE.size src, dst</code>	byte, word, long	4	N, Z, V (0), C (0)	Copies a value from <code>src</code> to <code>dst</code> .
<code>MOVEA</code>	<code>MOVEA.size src, An</code>	word, long	4	none	Loads an address register. Not a separate opcode — a <code>MOVE</code> whose destination is <code>An</code> is <code>MOVEA</code> , bit-for-bit, which is exactly why it skips the flags a plain <code>MOVE</code> would set. Byte size raises the Illegal Instruction exception .
<code>MOVEQ</code>	<code>MOVEQ #data, Dn</code>	long	4	N, Z, V (0), C (0)	Loads a small immediate (-128 to 127) into a data register. Faster/shorter than <code>MOVE.L #imm, Dn</code> .

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
LEA	LEA src, An	long	4	none	Computes an address and loads it into An , without reading what's stored there. Same addressing modes as JSR — see below .
PEA	PEA src	long	12	none	Like LEA , but pushes the address onto the stack instead of loading it into a register. Same addressing modes as JSR — see below .
SWAP	SWAP Dn	long	4	N, Z, V (0), C (0)	Swaps the high and low 16-bit halves of Dn .
EXG	EXG Dx, Dy / EXG Ax, Ay / EXG Dx, Ay	long	6	none	Swaps two full 32-bit registers — any mix of data and address registers.
MOVEM	MOVEM.size list, dst / MOVEM.size src, list	word, long	see below	none	Moves any subset of the 16 registers to or from memory at once, picked by a bitmask. See below for the addressing modes, the bitmask order, and the cycle formula.
MOVEP	MOVEP.size Dx, (d16, Ay) / MOVEP.size (d16, Ay), D x	word, long	16 (word), 24 (long)	none	Transfers a data register to/from alternating bytes of memory, for talking to an 8-bit peripheral over the 16-bit bus. See below for exactly which bytes and in what order.

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
<code>MOVE SR</code>	<code>MOVE SR, dst</code>	word	6 (<code>Dn</code>), 8 (mem)	none	Writes the 5 flags packed as a word (<code>x</code> at bit 4 down to <code>c</code> at bit 0) into <code>dst</code> . The high byte real hardware would report (supervisor bit, interrupt mask, trace bit) is always <code>0</code> here — see below . Not privileged on the real MC68000 this emulator targets, unlike <code>MOVE to SR</code> .
<code>MOVE to CCR</code>	<code>MOVE src, CCR</code>	word	12	X, N, Z, V, C	Reads a word from <code>src</code> and sets the 5 flags from its low 5 bits, ignoring the rest (see What are SR and CCR?). Never privileged on any 68000-family part.

Arithmetic

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
<code>ADD</code>	<code>ADD.size src, Dn / ADD.size Dn, dst</code>	byte, word, long	4 (Dn), 8/12 byte- word/ long (mem)	N, Z, V, C, X	Adds <code>src</code> into a data register, or a data register into memory — see below for the second form's addressing restriction and why the cycle count is higher.

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
ADDI	ADDI.size #data, dst	byte, word, long	8/16 byte- word/ long (Dn), 16/28 (mem)	N, Z, V, C, X	A different opcode from <code>ADD #imm, Dn</code> above — adds an immediate directly into <code>dst</code> (<code>Dn</code> or memory) with no register on the source side at all; see below for why both exist. An address register isn't a valid <code>dst</code> — that raises the Illegal Instruction exception .
ADDA	ADDA.size src, An	word, long	8 (word), 6 (long)	none	<code>ADD</code> 's <code>An</code> -destination form: always a full 32-bit add, word <code>src</code> sign-extended first. Same relationship <code>MOVEA</code> has to <code>MOVE</code> — no flags touched at all, not even the ones a same-size <code>ADD</code> would set.
ADDX	ADDX Dy, Dx / ADDX - (Ay), - (Ax)	byte, word, long	4/8 byte- word/ long (register), 18/30 (memory)	N, Z (see below), V, C, X	<code>ADD</code> 's extend-carry sibling: <code>dst = dst + src + X</code> , for chaining an addition wider than one register — the binary counterpart to <code>ABCD</code> 's decimal chaining. See below for the chaining idiom and the <code>Z</code> rule (identical here).

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
SUB	<code>SUB.size</code> <code>src, Dn /</code> <code>SUB.size</code> <code>Dn, dst</code>	byte, word, long	4 (Dn), 8/12 byte- word/ long (mem)	N, Z, V, C, X	Subtracts <code>src</code> from a data register, or a data register from memory — same two directions <code>ADD</code> has, see below .
SUBI	<code>SUBI.size</code> <code>#data, dst</code>	byte, word, long	8/16 byte- word/ long (Dn), 16/28 (mem)	N, Z, V, C, X	<code>ADDI</code> 's subtraction counterpart, same relationship <code>SUB</code> has to <code>ADD</code> — subtracts an immediate directly from <code>dst</code> , no register on the source side. Same <code>An</code> restriction as <code>ADDI</code> .
SUBA	<code>SUBA.size</code> <code>src, An</code>	word, long	8 (word), 6 (long)	none	<code>SUB</code> 's <code>An</code> -destination form — same rules <code>ADDA</code> follows.
SUBX	<code>SUBX</code> <code>Dy, Dx /</code> <code>SUBX -</code> <code>(Ay), - (Ax)</code>	byte, word, long	4/8 byte- word/ long (register), 18/30 (memory)	N, Z (see below), V, C, X	<code>SUB</code> 's extend-carry sibling: <code>dst = dst - src - X</code> , same relationship <code>ADDX</code> has to <code>ADD</code> .
ADDQ / SUBQ	<code>ADDQ</code> <code>#data, dst /</code> <code>SUBQ</code> <code>#data, dst</code>	byte, word, long	4	N, Z, V, C, X (<code>An</code> : none)	Adds/subtracts a small immediate (1 - 8) straight into <code>dst</code> , packed into the opcode itself — see below for how. <code>dst = An</code> is always a full 32-bit op with no flags touched, regardless of size — same rule <code>MOVEA</code> follows.

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
CMP	<code>CMP.size src, Dn</code>	byte, word, long	4	N, Z, V, C	Subtracts <code>src</code> from a data register like <code>SUB</code> , but only sets flags — the register itself is unchanged. Typically followed by a <code>Bcc</code> .
CMPI	<code>CMPI.size #data, dst</code>	byte, word, long	8/14 byte- word/ long (<code>Dn</code>), 12/20 (mem)	N, Z, V, C	<code>CMP</code> 's immediate counterpart — compares an immediate directly against <code>dst</code> (<code>Dn</code> or memory), only sets flags, same as <code>CMP</code> . Flag <code>X</code> untouched. Same <code>An</code> restriction as <code>ADDI</code> .
CMPM	<code>CMPM (Ay)+, (Ax)+</code>	byte, word, long	12/20 byte- word/ long	N, Z, V, C	<code>CMP</code> 's memory-to-memory form — no register involved, both addresses postincrement. Computes <code>(Ax) - (Ay)</code> , only sets flags. Flag <code>X</code> untouched.
CMPA	<code>CMPA.size src, An</code>	word, long	6	N, Z, V, C	<code>CMP</code> 's <code>An</code> -destination form: compares the full 32-bit <code>An</code> against <code>src</code> (sign-extended if word), without modifying <code>An</code> .
CLR	<code>CLR.size dst</code>	byte, word, long	4	N, Z, V (0), C (0)	Sets <code>dst</code> to 0. <code>An</code> address register isn't a valid <code>dst</code> — that raises the Illegal Instruction exception .

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
NEG	<code>NEG.size dst</code>	byte, word, long	4	N, Z, V, C, X	Negates <code>dst</code> in place (two's complement: <code>dst = 0 - dst</code>). An address register isn't a valid <code>dst</code> — that raises the Illegal Instruction exception.
NEGX	<code>NEGX.size dst</code>	byte, word, long	4	N, Z (see below), V, C, X	<code>NEG</code> 's extend-carry sibling, <code>ADDX / SUBX</code> 's single-operand relative: <code>dst = 0 - dst - X</code> .
TST	<code>TST.size dst</code>	byte, word, long	4	N, Z, V (0), C (0)	Sets flags from <code>dst</code> , like <code>CMP.size #0, dst</code> — doesn't modify it. An address register isn't a valid <code>dst</code> — that raises the Illegal Instruction exception.
TAS	<code>TAS dst</code>	byte	4 (<code>Dn</code>), 14 (memory)	N, Z (from the value read), V (0), C (0)	Like <code>TST.B</code> , but also sets <code>dst</code> 's bit 7 to 1 afterward — a "busy" flag for a spinlock. See below for the idiom.
EXT	<code>EXT.size Dn</code>	word, long	4	N, Z, V (0), C (0)	Sign-extends <code>Dn : .W</code> extends the low byte into the low word (high word untouched); <code>.L</code> extends the low word into the full long.
MULU	<code>MULU.W src, Dn</code>	word (source)	70	N, Z, V (0), C (0)	Unsigned multiply: <code>Dn = src × Dn.W</code> , full 32-bit result in <code>Dn</code> .

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
MULS	MULS.W src, Dn	word (source)	71	N, Z, V (0), C (0)	Signed multiply: $Dn = src \times Dn.W$, full 32-bit result in Dn .
DIVU	DIVU.W src, Dn	word (source)	138 (10 on overflow, 38 on zero divide)	N, Z, V, C (0)	Unsigned divide: Dn (32-bit) $\div$ src (16-bit) $\rightarrow$ quotient in Dn 's low word, remainder in the high word. If the quotient doesn't fit in 16 bits, V is set and Dn is left unmodified. Dividing by zero raises the Zero Divide exception instead.
DIVS	DIVS.W src, Dn	word (source)	158 (10 on overflow, 38 on zero divide)	N, Z, V, C (0)	Signed divide, same layout as <code>DIVU</code> . Truncates toward zero; the remainder takes the dividend's sign.

Binary Coded Decimal

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
ABCD	ABCD Dy, Dx / ABCD - (Ay), - (Ax)	byte	6 (register), 18 (memory)	X, C, Z (see below); N, V undefin ed	Adds two packed-BCD digits plus x : $Dx = Dx + Dy + x$. See below for what "packed BCD" means and why the flags behave differently here.

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
SBCD	SBCD Dy, Dx / SBCD - (Ay), - (Ax)	byte	6 (register), 18 (memory)	X, C, Z (see below); N, V undefined	Subtracts two packed-BCD digits plus X: $Dx = Dx - Dy - X$.
NBCD	NBCD dst	byte	6 (Dn), 8 (memory)	X, C, Z (see below); N, V undefined	Negates a packed-BCD byte plus X: $dst = 0 - dst - X$. An direct isn't valid — see below .

Logical

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
AND	AND.size src, Dn / AND.size Dn, dst	byte, word, long	4 (Dn), 8/12 byte- word/ long (mem)	N, Z, V (0), C (0)	Bitwise ANDs src into a data register, or a data register into memory — see below .
ANDI	ANDI.size #data, dst	byte, word, long	8/16 byte- word/ long (Dn), 16/28 (mem)	N, Z, V (0), C (0)	A different opcode from AND #imm, Dn above — ANDs an immediate directly into dst (Dn or memory) with no register on the source side. Same An restriction as ADDI (see Arithmetic).

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
ANDI to CCR	ANDI #data, CCR	byte	20	X, N, Z, V, C	A fixed opcode ANDI repurposes for <ea> = #imm, CCR — ANDs an immediate directly into the flags, instead of into a memory/register dst . See below .
OR	OR.size src, Dn / OR.size Dn, dst	byte, word, long	4 (Dn), 8/12 byte- word/ long (mem)	N, Z, V (0), C (0)	Bitwise ORs src into a data register, or a data register into memory — see below .
ORI	ORI.size #data, dst	byte, word, long	8/16 byte- word/ long (Dn), 16/28 (mem)	N, Z, V (0), C (0)	ANDI 's OR counterpart — ORs an immediate directly into dst , no register on the source side. Same An restriction as ADDI .
ORI to CCR	ORI #data, CCR	byte	20	X, N, Z, V, C	ANDI to CCR 's OR counterpart — ORs an immediate directly into the flags.
XOR	XOR.size Dn, dst	byte, word, long	4	N, Z, V (0), C (0)	Bitwise XORs a data register into dst — the one bitwise op where the source is always Dn and dst can be memory.

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
EORI	EORI.size #data, dst	byte, word, long	8/16 byte- word/ long (Dn), 16/28 (mem)	N, Z, V (0), C (0)	XOR 's immediate counterpart (real mnemonic EOR) — XORs an immediate directly into dst , no register on the source side. Same An restriction as ADDI .
EORI to CCR	EORI #data, CCR	byte	20	X, N, Z, V, C	ANDI to CCR 's XOR counterpart — XORs an immediate directly into the flags.
NOT	NOT.size dst	byte, word, long	4	N, Z, V (0), C (0)	Bitwise inverts dst in place (one's complement: dst = ~dst). An address register isn't a valid dst — that raises the Illegal Instruction exception .

Bit Manipulation

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
BTST	BTST #n, dst / BTST Dn, dst	long (register), byte (memory)	see below	Z only	Tests bit <code>n</code> of <code>dst</code> (Z=1 when clear). Doesn't modify <code>dst</code> — the standard way to poll one button out of the gamepad bitmask . <code>Dn, dst</code> reads the bit number from a register instead of an immediate. See below for the address-register restriction and the static/dynamic cycle difference.
BCHG	BCHG #n, dst / BCHG Dn, dst	long (register), byte (memory)	see below	Z only	Like <code>BTST</code> , but also toggles bit <code>n</code> of <code>dst</code> after testing it. See below for what "write back" means here.
BCLR	BCLR #n, dst / BCLR Dn, dst	long (register), byte (memory)	see below	Z only	Like <code>BTST</code> , but also clears bit <code>n</code> of <code>dst</code> after testing it. See below .
BSET	BSET #n, dst / BSET Dn, dst	long (register), byte (memory)	see below	Z only	Like <code>BTST</code> , but also sets bit <code>n</code> of <code>dst</code> after testing it. See below .

Shift and Rotate

`Dn` shifted/rotated in place by either an immediate count (`#n`, 1-8, with `#0` meaning 8) or a dynamic count taken from another data register (mod 64). There's also a second form that shifts/rotates a memory `<ea>` directly, always by exactly one bit — see [below](#) for its addressing modes, size restriction, and cycle cost.

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
ASL	ASL #n, Dn / Dx, Dn	byte, word, long	6 + 2×count	N, Z, V, C, X	Arithmetic shift left. Sets V if the sign bit changes value at any point during the shift.
ASR	ASR #n, Dn / Dx, Dn	byte, word, long	6 + 2×count	N, Z, V (0), C, X	Arithmetic shift right — copies the original sign bit back in at each step.
LSL	LSL #n, Dn / Dx, Dn	byte, word, long	6 + 2×count	N, Z, V (0), C, X	Logical shift left, filling with 0.
LSR	LSR #n, Dn / Dx, Dn	byte, word, long	6 + 2×count	N, Z, V (0), C, X	Logical shift right, filling with 0.
ROL	ROL #n, Dn / Dx, Dn	byte, word, long	6 + 2×count	N, Z, V (0), C	Rotates left — the bit rotated out of the top wraps back into bit 0. X is never touched.
ROR	ROR #n, Dn / Dx, Dn	byte, word, long	6 + 2×count	N, Z, V (0), C	Rotates right — the bit rotated out of bit 0 wraps back into the top. X is never touched.
ROXL	ROXL #n, Dn / Dx, Dn	byte, word, long	6 + 2×count	N, Z, V (0), C, X	Rotates left <i>through</i> X — the bit rotated out becomes the new X / C ; the <i>old</i> X rotates in where ROL would wrap the outgoing bit. See below .
ROXR	ROXR #n, Dn / Dx, Dn	byte, word, long	6 + 2×count	N, Z, V (0), C, X	Rotates right <i>through</i> X , mirroring ROXL .

(A dynamic count of 0 — only possible with the Dx, Dn form — does nothing to the value. For every instruction here except ROXL / ROXR : C comes out

cleared, `X` is left exactly as it was. `ROXL` / `ROXR` are the one exception — see [below](#).)

Program Control

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
<code>BRA</code>	<code>BRA target</code>	word	10	none	Always jumps to <code>target</code> .
<code>Bcc</code>	see below	word	10	none (reads flags, doesn't set them)	Jumps to <code>target</code> only if the named condition on the current flags holds.
<code>JMP</code>	<code>JMP target</code>	word	8	none	Jumps to <code>target</code> unconditionally — no return address is pushed. See below for which addressing modes are valid.
<code>JSR</code>	<code>JSR target</code>	word	16	none	Pushes the return address onto the stack, then jumps to <code>target</code> . See below for which addressing modes are valid.
<code>BSR</code>	<code>BSR target</code>	word	18	none	Like <code>JSR</code> , but PC-relative — pushes the return address , then always branches to <code>target</code> .
<code>RTS</code>	<code>RTS</code>	word	16	none	Pops a return address pushed by <code>JSR</code> / <code>BSR</code> and jumps there.

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
RTR	RTR	word	20	X, N, Z, V, C	RTR's sibling: pops a 16-bit word into the flags first (only the low 5 bits are used), then pops a return address and jumps there, same as RTS. Not tied to exceptions or supervisor mode — see below .
DBcc	DBcc Dn, target	word	10 / 12 / 14	none	Tests condition cc (same table as Bcc), then either stops or loops back to target. See below for exactly how, and what the three cycle counts mean.
Scc	Scc dst	byte	4 / 6 / 8	none	Tests condition cc (same table as Bcc) and sets dst to \$FF or \$00 — no branch, no arithmetic. See below for valid destinations and what the three cycle counts mean.
LINK	LINK An, #displacement	word	16	none	Stack-frame prologue: pushes An, points An at the new frame, then moves SP by displacement. See below for the LINK A7 / UNLK A7 special case.

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
UNLK	UNLK An	word	12	none	Stack-frame epilogue, <code>LINK</code> 's inverse: restores <code>SP</code> from <code>An</code> , then pops the old <code>An</code> value. See below for the <code>UNLK A7</code> special case.
CHK	CHK <ea>, Dn	word	10 / 40	N (see below)	Bounds-checks <code>Dn</code> against <code>0</code> and <code><ea></code> (both signed words); out of range either way raises the CHK exception instead of continuing. See below for the exact range and what happens to <code>N</code> .

System

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
NOP	NOP	word	4	none	Does nothing. Useful for timing/padding.
ILLEGAL	ILLEGAL	word	34	none	Deliberately raises the Illegal Instruction exception — a reserved opcode, useful as a portable, explicit "trap here" marker.

Mnemonic	Syntax	Sizes	Cycles	Flags affected	Description
TRAPV	TRAPV	word	4 (V clear), 34 (V set)	none (reads V, doesn't set it)	Raises the TRAPV exception if <code>v</code> is set, otherwise falls through — checks for overflow after an <code>ADD / SUB</code> without a separate <code>BVC / TRAP</code> pair.

Detailed explanations, restrictions and examples for individual instructions live in [Instruction Usage Guidance](#).

See [TRAP System Calls](#) below for `TRAP`, and [below](#) for why `ILLEGAL / TRAPV` are here but `RTE / STOP / RESET / MOVE SR` aren't.

Alphabetical Index

A — [ABCD](#) · [ADD](#) · [ADDA](#) · [ADDI](#) · [ADDQ](#) · [ADDX](#) · [AND](#) · [ANDI](#) · [ANDI to CCR](#) · [ASL](#) · [ASR](#)

B — [Bcc](#) · [BCHG](#) · [BCLR](#) · [BRA](#) · [BSET](#) · [BSR](#) · [BTST](#)

C — [CHK](#) · [CLR](#) · [CMP](#) · [CMPA](#) · [CMPI](#) · [CMPM](#)

D — [DBcc](#) · [DIVS](#) · [DIVU](#)

E — [EORI](#) · [EORI to CCR](#) · [EXG](#) · [EXT](#)

I — [ILLEGAL](#)

J — [JMP](#) · [JSR](#)

L — [LEA](#) · [LINK](#) · [LSL](#) · [LSR](#)

M — [MOVE](#) · [MOVEA](#) · [MOVEM](#) · [MOVEP](#) · [MOVEQ](#) · [MOVE SR](#) · [MOVE to CCR](#) · [MULS](#) · [MULU](#)

N — [NBCD](#) · [NEG](#) · [NEGX](#) · [NOP](#) · [NOT](#)

O — [OR](#) · [ORI](#) · [ORI to CCR](#)

P — [PEA](#)

R — [ROL](#) · [ROR](#) · [ROXL](#) · [ROXR](#) · [RTR](#) · [RTS](#)

S — [SBCD](#) · [Scc](#) · [SUB](#) · [SUBA](#) · [SUBI](#) · [SUBQ](#) · [SUBX](#) · [SWAP](#)

T — [TAS](#) · [TRAP](#) · [TRAPV](#) · [TST](#)

U — UNLK

X — XOR

Instruction Usage Guidance

Answers to the "how does this actually behave?" questions the tables above can't fit in a cell: operand directions, flag and register side effects, which addressing modes an instruction accepts, and worked examples. The instruction tables and the alphabetical index stay in [Instruction Set \(Opcodes\)](#).

How does the memory-destination direction work?

`ADD`, `SUB`, `AND`, and `OR` each support two directions that share one mnemonic (see [Addressing Modes](#) above for what `<ea>` means):

- `<ea>, Dn` — read a value from anywhere (any addressing mode, including `#imm`) and combine it into a data register. This is the form used everywhere else on this page.
- `Dn, <ea>` — the mirror image: combine a data register's value into `<ea>` instead, writing the result back to `<ea>` rather than to `Dn`. `<ea>` here must be a *memory-alterable* address — not `Dn`, not `An`, no `#imm`, no PC-relative — the same restriction the [memory-operand shift/rotate form](#) uses.

```
MOVE.B    #5, D0
ADD.B     D0, (A1)    ; Memory[A1] += D0, not D0 += Mem[A1]
```

The memory-destination form costs more than the flat 4 cycles the `<ea>, Dn` direction uses: 8 for byte or word, 12 for long — writing the result back to memory is a real extra bus cycle the register-destination form doesn't pay.

`AND`'s and `OR`'s `Dn, <ea>` form has one more wrinkle worth knowing if you're hand-encoding opcodes: their byte-sized `mode 000 / 001` slot is reserved on real hardware for `ABCD` (in `AND`'s case) or `SBCD` (in `OR`'s case) — see [How does packed BCD arithmetic work?](#). This isn't a conflict in practice: `Dn, <ea>`'s own `<ea>` already excludes `Dn / An` (mode `000 /`

001), so the two instructions never actually compete for the same encoding — the split just happens to land exactly where `Dn, <ea>` was never going to use anyway.

Why does `ADDI` exist when `ADD #imm, Dn` already works?

Both really do accept an immediate — `ADD`'s `<ea>, Dn` form's `src` can be any addressing mode, `#imm` included (see [above](#)), so `ADD #5, D0` is a perfectly ordinary `ADD`, not `ADDI`. `ADD` / `ADDI` are still genuinely different opcodes at the bit level, and the difference that actually matters is `dst`: `ADD #imm, Dn`'s `<ea>, Dn` form can only ever target a data register, while `ADDI #imm, dst` can also target memory directly. So reach for `ADDI` specifically when the immediate needs to land straight in memory, with no register involved at all:

```
ADDI.W #1, (A0) ; adds 1 straight into memory
```

`SUBI` / `ANDI` / `ORI` / `EORI` / `CMPI` all follow the same split from their non-`I` counterpart.

How does `ADDQ`/`SUBQ` pack its immediate into the opcode?

`ADDQ` / `SUBQ`'s immediate is restricted to 1 - 8 for a specific reason: that narrow range is exactly what fits in the 3 spare bits (`ddd`) the opcode word already has free, right alongside the size and addressing-mode bits — `0101ddd0ssmmrrr` for `ADDQ`, `0101ddd1ssmmrrr` for `SUBQ`. Unlike `ADDI` / `SUBI`, which read their immediate from a separate extension word fetched right after the opcode, `ADDQ` / `SUBQ` never fetch one at all — the value is already sitting in the opcode word the CPU just read to know *which* instruction this is. That's also part of why they're cheaper: see their 4-cycle entry in the [Arithmetic](#) table above, versus `ADDI` / `SUBI`'s.

3 bits can only encode 0 - 7, though, and the instructions need 1 - 8 — so 8 reuses the otherwise-unused `000` bit pattern: the assembler encodes an immediate of 8 as `000`, and the CPU decodes a `000` field

back as `8`, never as `0`. There's simply no encoding for "add/subtract `0`" with these two instructions:

ddd bits	Immediate value
<code>001</code>	1
<code>010</code>	2
<code>011</code>	3
<code>100</code>	4
<code>101</code>	5
<code>110</code>	6
<code>111</code>	7
<code>000</code>	8 (not 0)

This is a deliberate design decision made by Motorola's engineers when they defined the 68000 instruction set, not something specific to this emulator: adding or subtracting `0` is a no-op, so encoding it would have wasted one of the 8 possible `ddd` patterns on a value nobody would ever want. Reusing `000` for `8` instead stretches the same 3 bits to cover the actual useful range, `1 - 8`.

```
MOVEQ    #0,D0
ADDQ.L   #8,D0      ; D0 = 8 -- "000" decodes as 8, not 0
```

How does TAS work as a lock?

`TAS dst` does two things in one instruction: it sets flags from `dst` exactly like `TST.B dst` would (`N` from the value's sign bit, `Z` if it was `0`), then — regardless of what it just read — forces `dst`'s bit 7 to `1` and writes that back. Reading the old value and setting the new one happen as a single indivisible step on real 68000 hardware, which is the entire point: it's the classic building block for a *spinlock*, a busy-wait flag that only one caller can ever "win":

```

FLAG:   DS.B   1           ; the lock: a single reserved byte
        EVEN           ; DS.B 1 is odd-sized, see below
LOOP:
    TAS   FLAG           ; N = old bit 7, then sets FLAG's bit 7
    BMI   LOOP           ; N set: was already busy - spin
    ; N clear: lock acquired, critical section entered
    ...
    CLR.B FLAG           ; release the lock for the next caller

```

`FLAG` here is a [label](#) — a name the program itself gives to a byte of memory it reserved for the lock — not a keyword or a special register. The `EVEN` after it pads to the next even address, since `DS.B 1` reserves an odd number of bytes and `LOOP`'s first instruction needs to start on an even one — see [Source Layout](#).

If `FLAG`'s bit 7 was already `1`, `N` comes out set and the loop spins — someone else holds the lock. If it was `0`, `N` comes out clear, execution falls through, and `TAS` has *already* set the bit on its way out — no other caller can slip in between the test and the set, because they were never two separate steps to begin with.

This emulator has no concurrency (no threads, no interrupts preempting mid-instruction) to actually race against, so a plain read followed by a plain write already behaves identically to the indivisible version — the idiom above works the same way it would on real hardware, just without anything else that could ever contend for the lock.

An `direct` isn't a valid `dst` — there's no such thing as test-and-setting an address register — so it raises the [Illegal Instruction exception](#) instead, the same restriction `BTST / CHK` have on their own `<ea>`.

How does packed BCD arithmetic work?

Packed BCD stores two decimal digits, `0 - 9` each, one per nibble of a byte — a completely different interpretation of the bits than plain binary. `$09` means the decimal digit 9 either way, but `$99` is decimal 99 in packed BCD, not 153 like it would be read as plain binary. Adding `$09` and `$01` as ordinary binary gives `$0A` — not a valid pair of decimal digits — which is exactly the problem `ABCD` corrects for, producing `$10` instead: the same digits ("1", "0") you'd get adding 9 and 1 by hand and carrying.

```
MOVE.B  #$09,D0
MOVE.B  #$01,D1
ABCD    D0,D1      ; D1 = 9+1 = $10, not binary $0A
```

All three instructions are byte-only and thread `X` through as a carry/borrow, so a decimal number wider than one byte can be processed one byte at a time, low byte first — the same chaining idiom `ADDX / SUBX / NEGX` use for plain binary. `ABCD / SBCD`'s `-(Ay), -(Ax)` form exists specifically for this: it walks two multi-byte BCD numbers backward through memory together, one digit-pair at a time.

A real consequence of `NBCD` being defined as `0 - dst - X`: negating a zero byte with `X` already set doesn't stay zero — it borrows, producing `$99` with `C / X` set. That's not a bug, it's what makes negating a multi-byte BCD number work: negate the low byte first, then each higher byte's `NBCD` sees the previous byte's borrow via `X` and accounts for it.

Two flag quirks worth knowing before relying on them:

- **N and V are genuinely undefined** on real 68000 hardware for a BCD result — a packed-decimal byte's top bit isn't a sign bit, so there's no meaningful value to compute. This emulator leaves them untouched rather than inventing one, the same choice `CHK` makes for its own undefined flags.
- **Z is cleared if the result is non-zero, but left alone if the result is zero** — not a plain assignment like every other instruction on this page. That's deliberate: it lets a multi-byte chain clear `Z` once before the first byte, then read `Z=1` at the end only if *every* byte in the chain came out zero, without each individual byte's instruction able to falsely set `Z` back to `1` on its own.

`NBCD`'s `dst` follows the same restriction `BTST / CHK / TAS` place on their own operands: An direct raises the [Illegal Instruction exception](#) instead of being treated as a value to negate.

What do BCHG/BCLR/BSET write back?

Same as `BTST` — the bit is located and its old value drives `Z` — but instead of stopping there, the (possibly changed) value gets written back to `dst`: `BCHG` flips the bit, `BCLR` forces it to `0`, `BSET` forces it to `1`. `Z`

still reflects the bit's state *before* the write, exactly like `BTST`, so `BSET #0,D0 / BEQ WAS_CLEAR` reads naturally: branch if the bit *used to be* clear, even though it's `1` now.

An `direct` isn't a valid `dst` for any of the four (`BTST` included) — same restriction `CHK / TAS / NBCD` share — so the static (`#n,dst`) form raises the [Illegal Instruction exception](#) instead. The dynamic (`Dn,dst`) form's own `An`-direct opcode slot belongs to `MOVEP` on real hardware instead (see [Data Movement](#)), so hand-encoding e.g. `BSET D0,A0` runs as `MOVEP`, not as a failed `BSET`.

Static vs dynamic bit number: `#n,dst` reads the bit number from an immediate extension word; `Dn,dst` reads it from a data register instead. Behavior is otherwise identical (same `Z` rule, same write-back, same long-register/byte-memory split) — the only difference is cost, and only when `dst` is itself a register (no extension word to fetch):

Instruction	Static, register dst	Dynamic, register dst	Either form, memory dst
<code>BTST</code>	10	6	8
<code>BCHG</code>	12	8	12
<code>BCLR</code>	14	10	12
<code>BSET</code>	12	8	12

`BCLR`'s register-form cycle count is genuinely higher than `BCHG / BSET`'s on real 68000 hardware in both forms — not a typo.

Which Bcc do I want?

After a `CMP`, there are *two separate* families of "is it bigger/smaller" branches — one for **signed** numbers (can be negative), one for **unsigned** (always treated as a positive count). Mixing them up is a classic bug: comparing `$FFFFFFFF` to `1`, as signed that's `-1 < 1` (`BLT` is true), but as unsigned `$FFFFFFFF` is a huge number `> 1` (`BHI` is true) — same bits, opposite answer. Pick the family that matches what the value actually

represents (a loop counter is usually unsigned; a temperature could be signed).

Signed comparisons (`CMP.L #5, D0` then...):

Mnemonic	Branches when	Meaning
BGT	signed >	Greater Than
BGE	signed >=	Greater or Equal
BLT	signed <	Less Than
BLE	signed <=	Less or Equal

Unsigned comparisons (same syntax, different meaning of "bigger"):

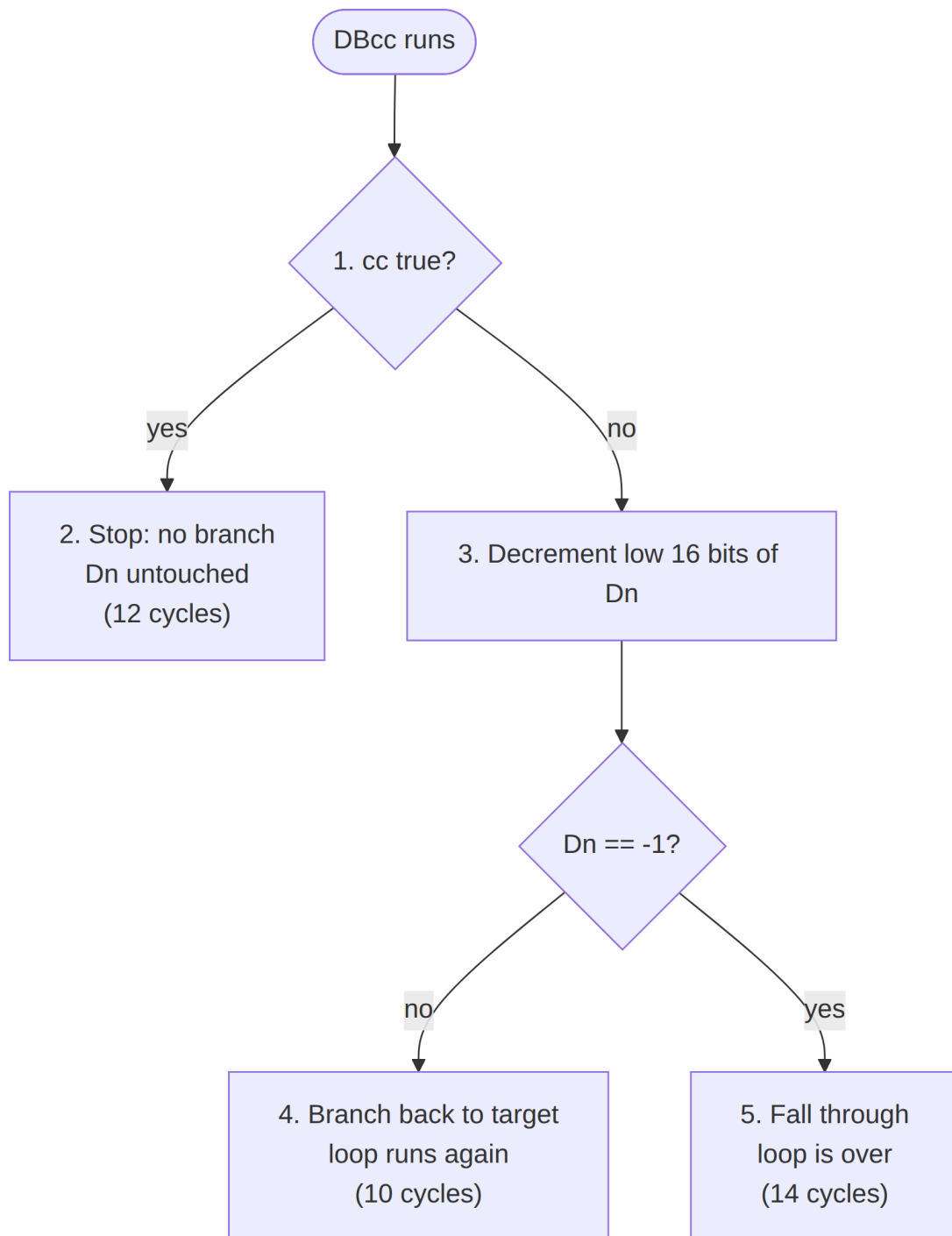
Mnemonic	Branches when	Meaning
BHI	unsigned >	Higher
BCC (= BHS)	unsigned >=	Carry Clear / Higher or Same
BCS (= BLO)	unsigned <	Carry Set / Lower
BLS	unsigned <=	Lower or Same

Equality and raw flag tests (same either way):

Mnemonic	Branches when	Meaning
BEQ	Z=1 (equal)	Equal
BNE	Z=0 (not equal)	Not Equal
BPL	N=0	Plus (result was positive/zero)
BMI	N=1	Minus (result was negative)
BVC	V=0	Overflow Clear
BVS	V=1	Overflow Set

How does DBcc decide?

`target` works the same way it does for `Bcc` : a label placed earlier in the code, so branching back to it repeatedly is what forms a loop — there's no loop construct in the CPU itself, just this instruction deciding, each time it runs, whether to jump back or not.



Spelled out, step by step:

1. Test condition `cc` (the same table as `Bcc` , above).
2. **Already true?** Stop here: no branch, `Dn` untouched. (12 cycles.)
3. **False** — decrement the low 16 bits of `Dn` .
4. **Result isn't -1 ?** Branch back to `target` : the loop runs again. (10 cycles.)
5. **Result is -1 ?** Fall through instead: the loop is over. (14 cycles.)

`DBRA` (a.k.a. `DBF`) is `DBcc` with `cc` fixed to "always false" — step 2 never fires, so it always falls through to the decrement. See the `DBRA` example further down this page, including the classic off-by-one it's easy to trip over.

Which destinations can `Scc` use?

Any data-alterable destination — `Dn` or writable memory, same restriction `CLR / NOT / NEG / TST` use. Not valid: `An` , `#value` , or a PC-relative address (`d16(PC)` , `d8(PC,Xn)`).

The three cycle counts depend on both the destination and the outcome of testing `cc` : 4 cycles for `Dn` when `cc` turns out false, 6 when it's true, 8 for a memory destination either way.

What is a "return address"?

The address of the instruction right after the call — where execution should pick back up once the called code is done. `JSR / BSR` push it onto the stack when they jump; `RTS` pops it back off and jumps there, resuming exactly where the caller left off. This is the 68000's version of a `call / return` mechanism.

Concretely, `PC` (the [Program Counter](#)) already points past an instruction by the time that instruction runs — the CPU advances it during fetch, before execution. So when `JSR` pushes "the return address", it's just pushing `PC` 's current value at that point, which happens to be right after the `JSR` instruction (and past any extension word its addressing mode used, e.g. the `d16` in `JSR $10(A0)`):

```
JSR    MY_FUNC        ; at address $2000, 2 bytes long
; <-- $2002 is what gets pushed as the return address
```

Inside `MY_FUNC`, a final `RTS` pops that `$2002` back into `PC`, so execution resumes right after the call. `JMP` skips all of this — no address is pushed, so there's nothing to return to.

Which addressing modes can JSR target?

Any that name a memory location without a register side effect — the 68000's "control" addressing modes: `(An)`, `d16(An)`, `d8(An,Xn)`, `xxx.W`, `xxx.L`, `d16(PC)`, `d8(PC,Xn)`. Not valid: `Dn`, `An`, `(An)+`, `-(An)`, or `#imm`. See [Addressing Modes](#) above for what each of these means (that section covers what every other instruction's `src / dst` can be, too). `JMP`, `LEA`, and `PEA` all use this exact same set for their `target / src` — they compute the address the same way `JSR` does, just jump to it directly (`JMP`, without pushing a return address), load it into `An` (`LEA`), or push it onto the stack (`PEA`) instead.

How do LINK and UNLK handle A7?

`LINK An,#displacement` sets up a stack frame in three steps: push `An` onto the stack, point `An` at that pushed value (the new frame pointer), then move `SP` by `displacement` — a negative value reserves that many bytes of local variables below the frame. `UNLK An` tears it back down in the opposite order: `SP` is restored from `An`, and the old `An` value is popped back off the stack. Together they're the standard prologue/epilogue for a subroutine that needs local variables:

```
MY_FUNC:
    LINK    A6,#-8          ; A6 = frame pointer, 8 bytes of locals
    MOVE.L  D0,-8(A6)       ; use the locals via A6
    ...
    UNLK    A6              ; restore SP, pop the caller's A6 back
    RTS
```

Both instructions are defined as an exact sequence of micro-operations, not one atomic "swap `An` and `SP`" step — `LINK`: `SP-4→SP`, `An→(SP)`, `SP←An`, `SP+displacement→SP`; `UNLK`: `SP←An`, `An←(SP)`, `SP←SP+4`. That only matters when `An` is `A7` (`LINK A7,#d` / `UNLK A7`), since then every step in the sequence reads or writes the same physical register the previous step just changed:

- **LINK A7, #d** : step 1 already decremented **A7** before step 2 pushes it, so the value that lands on the stack is the *new*, already-decremented **SP** — not **A7** 's value from before the instruction ran.
- **UNLK A7** : step 1 (**SP←An**) is a no-op, since **An** already is **SP** . Step 2 (**An←(SP)**) then overwrites **A7** with the popped value — so by the time step 3 runs, **SP←SP+4** adds 4 to the *popped* value, not to the original frame pointer.

Neither case comes up in the **LINK A6, #-8** / **UNLK A6** idiom above, since a subroutine almost always frames a different register than **SP** — but hand-encoding raw opcode words makes it easy to reach for **A7** by mistake, so it's worth knowing the sequence rather than assuming "swap" semantics.

Which values does CHK accept?

CHK <ea>, Dn treats **<ea>** as an upper bound and checks whether **Dn** 's low 16 bits, read as a signed value, falls in the range **0** to **<ea>** inclusive — the classic use is validating an array index before using it:

```
MOVE.W D3,D0      ; candidate index, computed earlier
CHK      #99,D0    ; valid range 0-99 (100-entry array)
; only reached if D0 was in 0..99 - safe to use as an index below
LEA      TABLE,A0
MOVE.W (A0,D0.W),D1 ; safe: D0 passed the bounds check
```

Two ways to fail, and the **N** flag tells you which one happened right before the exception fires:

- **Dn is negative** — **N** is set to **1** .
- **Dn is greater than the bound** — **N** is cleared to **0** .

Either failure raises the **CHK exception** (vector **\$48**) instead of falling through to the next instruction — same mechanism as **DIVU** / **DIVS** 's Zero Divide, just a different vector. When **Dn** is in range, execution just continues and **N** is left exactly as it was; **Z** , **V** , and **C** are undefined on real 68000 hardware in every case, so this emulator leaves them untouched too rather than picking an arbitrary value for them.

An `direct` isn't a valid `<ea>` here — there's no such thing as bounds-checking against an address register — so it raises the [Illegal Instruction exception](#) instead, the same restriction `BTST` has on its destination.

The `10 / 40` cycle split isn't arbitrary: `10` is just the cost of fetching and comparing, in range or not. The other `30` cycles only get spent when `CHK` actually traps — the same exception-stacking work (pushing the return state, reading the vector, jumping to the handler) every other exception here pays for. That work alone, with nothing to check first, is exactly what `TRAP / ILLEGAL`'s flat `34` cycles, and `TRAPV`'s `34`-cycle trap-taken case, already charge for.

How does `MOVEM`'s register list work?

`MOVEM` moves any subset of the 16 registers (`D0 - D7` , `A0 - A7`) to or from memory in one instruction. The register list — written as a range/list like `D0-D2/A0` , meaning `D0` , `D1` , `D2` , and `A0` — gets packed into a 16-bit bitmask, one bit per register, that follows the opcode word. (The assembler builds that bitmask for you from the written list; for `-(An)` stores it reverses the bit order automatically.)

Which addressing modes are valid depends on which direction the data is moving:

- **Register list → memory** (`MOVEM.size list,dst`): the [control addressing modes](#) `JSR / LEA / PEA` use, plus predecrement (`-(An)`).
- **Memory → register list** (`MOVEM.size src,list`): the same control addressing modes, plus postincrement (`(An)+`).

`Dn` , `An` direct, and `#imm` are never valid — there's no single register or constant to move a whole register list to or from.

Two quirks worth knowing before hand-encoding one:

1. **Predecrement reverses the bit order.** For every addressing mode except `-(An)` , bit 0 of the mask is `D0` and bit 15 is `A7` . But `-(An)` decrements the address *before* each store, filling memory backward — so to keep the lowest address holding the lowest-numbered register (matching normal reading order), the bit order flips too: bit 0 becomes `A7` , bit 15 becomes `D0` . A real assembler handles this automatically from the `list, -(An)` syntax; hand-encoding it means reversing the bits yourself.

2. **Word-size loads sign-extend.** `MOVEM.W src, list` sign-extends each 16-bit value it reads to the full 32 bits of its register — unlike `MOVE.W`, which only overwrites the low word and leaves the high word alone.

`MOVEM.W list, dst` (storing) just writes each register's low 16 bits, no extension involved.

Cycles: $8 + 4n$ (register→memory, word) / $8 + 8n$ (long); $12 + 4n$ (memory→register, word) / $12 + 8n$ (long) — n is the number of registers actually transferred, not 16.

How does MOVEP transfer alternating bytes?

`MOVEP` moves data between `Dx` and a byte-oriented peripheral wired onto the 68000's 16-bit data bus at every *other* address — a hardware detail this emulator doesn't need to model (it's flat byte-addressable memory), but the instruction still transfers the bytes in the same order and positions real hardware would, in case a program relies on it.

Starting at `(d16, Ay)`, it reads/writes one byte, steps the address by 2, reads/writes the next byte, and so on — 2 (`.W`) or 4 (`.L`) bytes total, most-significant byte first:

```
MOVEM.L D0, $0(A0)
; byte 0 (D0 bits 31-24) -> Memory[A0+0]
; byte 1 (D0 bits 23-16) -> Memory[A0+2]
; byte 2 (D0 bits 15-8)  -> Memory[A0+4]
; byte 3 (D0 bits 7-0)   -> Memory[A0+6]
```

`.W` only ever touches `Dx`'s low 16 bits — loading leaves the high word alone (same merge rule a plain word `MOVE` into `Dn` follows), storing only reads the low word out.

Unlike every other data-movement instruction, there's no addressing-mode field to pick from: `(d16, Ay)` is the only form `MOVEP` supports, always with a displacement (even `$0`, as above) — there's no `(An)`-only shorthand.

How does the memory-operand shift and rotate form work?

Every `ASL / ASR / LSL / LSR / ROL / ROR / ROLX / RORX` in [Shift and Rotate](#) above also has a second form that shifts/rotates a memory `<ea>` directly instead of a `Dn` — always by exactly one bit (there's no room left in the opcode for a count once `<ea>` is encoded), and word-sized only (same reason — no size field left either).

Valid `<ea>`: `(An)`, `(An)+`, `-(An)`, `d16(An)`, `d8(An,Xn)`, `xxx.W`, or `xxx.L` — anything except `Dn`, `An` direct, `#imm`, or a PC-relative address (see [Addressing Modes](#)). `Dn` and `An` both raise the [Illegal Instruction exception](#) here: shifting a `Dn` is what the register form above is for, and there's no such thing as shifting an address register.

Cycles: 8, flat, regardless of addressing mode.

Example — shift a value already sitting in memory, without loading it into a register first:

```
ASL      (A0)                ; Memory[A0] <= 1, in place
```

How does rotating through X differ from a plain rotate?

`ROLX / RORX` rotate through the `X` flag instead of leaving it out of the loop the way `ROL / ROR` do — the bit that gets rotated out becomes the new `X` (and `C` — the two always end up equal here), but the bit that rotates back *in* is whatever `X` held *before* the instruction ran, not the bit that just left. A `ROL / ROR` never has this extra step: the outgoing bit wraps straight back in on its own.

One consequence worth knowing: a rotate count of `0` (only possible with the `Dx,Dn` form) still sets `C` to `X`'s value. Every other instruction in this family either leaves `C / X` alone (`ASL / ASR / LSL / LSR`) or clears `C` (`ROL / ROR`) when nothing actually shifted — `ROLX / RORX` are the one case where "nothing shifted" still changes a flag.

Example — the same rotate, with and without `X`:

```

; D0 = %0000_0010, X = 1
      ROL    #1,D0      ; -> %0000_0100 (bit 7 wraps in)
; ...reset D0 to %0000_0010...
      ROXL   #1,D0      ; -> %0000_0101 (old X wraps in)

```

Why doesn't this emulator implement RTE/STOP/RESET/MOVE SR?

Real 68000 hardware has two privilege levels, user and supervisor, controlled by a bit in the Status Register (SR) — a 16-bit register this emulator only partially models. SR's low byte is the [Status Flags](#) every arithmetic/logic instruction on this page reads or sets (N / Z / V / C, plus X). SR's high byte — the supervisor bit, an interrupt priority mask, a trace bit — doesn't exist here at all, and neither does the separate supervisor stack pointer (SSP) real hardware switches to alongside it: raising an [exception](#) here only ever pushes PC, onto the one stack there is, A7.

That's a deliberate simplification, not an oversight: supervisor mode exists to protect a multi-program OS kernel from untrusted user code sharing one CPU. This emulator runs one program at a time with nothing to protect it from, so the privilege boundary has no job to do here. Concretely, that rules out:

- MOVE to SR, MOVE USP — genuinely privileged on real hardware, and

there's no separate USP register to move either.

- STOP — loads an immediate into SR (interrupt mask included) before

halting; the "immediate into SR" part has no home.

- RESET — pulses a hardware reset line to external peripherals; there's no peripheral bus to reset.

- RTE — pops SR and PC off the supervisor stack, possibly returning to user mode; this emulator's exceptions only ever push PC, so there's no SR for RTE to pop either.

`ILLEGAL` and `TRAPV` don't touch any of this — both are just alternate ways to *raise* an exception, through the exact same `PC`-only mechanism every other exception on this page already uses.

Correction worth being explicit about: `MOVE from SR` and `MOVE to CCR` are *not* privileged on the real MC68000 this emulator targets — `MOVE from SR` only became privileged starting with the 68010, and `MOVE to CCR / RTR` were never privileged at all. Both are implemented now (as `MOVE SR` and `MOVE to CCR` in the [Data Movement](#) table above), along with `CMPM` (see [Arithmetic](#)), `RTR` (see [Program Control](#)), and `ADDI / SUBI / ANDI / ORI / EORI / CMPI / ADDX / SUBX / NEGX` (see the [Arithmetic/Logical](#) tables), and `ANDI / ORI / EORI`'s own `#imm,CCR` special-case sub-forms (three fixed opcodes, e.g. `ORI #imm,CCR` at `$003C` — see `ANDI to CCR / ORI to CCR / EORI to CCR` in the [Logical](#) table) — every ordinary instruction that was ever on this page's gap list.

One more correction: an earlier version of this section also listed `MOVE from CCR` as missing — that instruction doesn't exist on the real MC68000 at all. Motorola only added it in the 68010, to give user programs a way to read the flags after `MOVE from SR` became privileged there. Nothing to implement.

`ANDI / ORI / EORI #imm,SR` (the privileged sibling of the now-implemented `#imm,CCR` forms) remains a genuine gap, alongside `MOVE to SR` above. Hand-encoding one today doesn't do anything meaningful — with no dedicated opcode entry of its own, it reaches `ANDI / ORI / EORI`'s own general form (no longer silently `MOVE`, that bug is fixed), which then throws a generic error instead of raising a catchable exception.

Worked Examples

Small programs to read alongside the instruction tables. All of them assemble. For complete, runnable programs, load one from the editor's "Load an example" menu.

Example — add two numbers and write a white pixel, using a direct absolute address:

```

MOVE.L  #100,D0
MOVE.L  #200,D1
ADD.L   D1,D0           ; D0 = 300
MOVE.L  #$FFFFFF,$40000 ; first pixel = white
TRAP    #0              ; exit

```

Example — a countdown loop using `MOVEQ`, `SUB`, `CMP`, and `Bcc`:

```

        MOVEQ   #5,D0           ; D0 = 5 (loop counter)
LOOP:
        MOVEQ   #1,D1
        SUB.L   D1,D0           ; D0 -= 1
        CMP.L   #0,D0
        BNE     LOOP           ; keep looping while D0 != 0
        TRAP    #0              ; exit

```

(Labels like `LOOP:` are handled by the assembler, which turns each `Bcc` target into the right byte offset for you. You can paste these snippets into the Editor and run them.)

Example — the same countdown, more idiomatically, using `DBRA`:

```

        MOVEQ   #4,D0           ; D0 = 4 (see gotcha below)
LOOP:
        ADD.W   #1,D1           ; (loop body - whatever it's for)
        DBRA    D0,LOOP         ; D0--, loop unless D0 hits -1
        TRAP    #0              ; exit

```

`DBRA` folds the decrement, the comparison, and the branch into one instruction — this is the loop `SUB.L / CMP.L / BNE` above builds by hand, in the form real 68000 code almost always uses instead. The classic gotcha, and the reason this loads `4` where the `Bcc` version above loaded `5`: the body always runs once *before* the first decrement, and the loop only stops once `Dn` has been decremented all the way to `-1`. So for the body to run exactly `N` times, `Dn` has to start at `N - 1`, not `N` — starting it at `5` here would run the body 6 times, not 5.

Example — calling a subroutine with `JSR / RTS` (again with a label, `DOUBLE:`):

```

    MOVEA.L #DOUBLE,A0          ; A0 = address of the subroutine
    MOVEQ   #21,D0
    JSR     (A0)                 ; D0 *= 2, then returns here
    TRAP    #0                   ; exit

DOUBLE:
    ADD.L   D0,D0                ; D0 += D0
    RTS     ; back to the caller

```

Example — saving and restoring registers around a subroutine call with `MOVEM`, so `DOUBLE` is free to use `D1` and `A0` as scratch without disturbing the caller's:

```

    MOVEQ   #1,D1
    MOVEA.L #$1000,A0
    MOVEM.L D1/A0,-(A7)          ; push D1,A0 (see above)
    MOVEA.L #DOUBLE,A0
    MOVEQ   #21,D0
    JSR     (A0)                 ; D0 *= 2; free to clobber D1/A0
    MOVEM.L (A7)+,D1/A0          ; restore, same order pushed
    TRAP    #0                   ; exit

DOUBLE:
    ADD.L   D0,D0
    RTS

```

TRAP System Calls

A `TRAP #n` instruction is a *software interrupt*: unlike `JSR / BSR`, which jump to an address you choose, `TRAP #n` jumps to one of 16 fixed, numbered entry points (`n = 0 - 15`, encoded right in the instruction itself) — think of it as calling a small built-in "operating system" routine by number instead of by address. On real 68000 hardware this mechanism is a blank slate: Motorola reserved the 16 vectors but assigned none of them a fixed job, so what `TRAP #5` (say) actually did depended entirely on whichever OS or ROM a given machine ran. Here, each of the 9 implemented vectors (`#0 - #8`) has one fixed, documented job — this table is effectively this emulator's own tiny "operating system calls" list.

Like any instruction that isn't a branch (`Bcc` , `JMP` , `JSR` ...), `TRAP #n` doesn't redirect execution anywhere: once it's done, the CPU simply moves on to whatever comes right after it in memory — the instruction written on the next line, unless a label or jump says otherwise. `TRAP #0` (halt) is the only exception, since it stops the CPU outright instead.

Vector	Syntax	Cycles	Description
#0	<code>TRAP #0</code>	4	Halts the CPU (ends the program).
#1	<code>TRAP #1</code>	4	Draws the null-terminated ASCII string at <code>A0</code> at pixel (<code>D0</code> , <code>D1</code>) in the <code>D2</code> color. See Printing Text with TRAP #1 below.
#2	<code>TRAP #2</code>	4	Reads the pixel at the framebuffer address in <code>A0</code> into <code>D0</code> (32-bit RGBA). See Addressing a Pixel for TRAP #2/#3 below.
#3	<code>TRAP #3</code>	4	Writes <code>D0</code> (32-bit RGBA) as the pixel at the framebuffer address in <code>A0</code> .
#4	<code>TRAP #4</code>	4	Fills every one of the 64,000 framebuffer pixels with <code>D0</code> (32-bit RGBA) — clears the screen to any solid color, not just black.
#5	<code>TRAP #5</code>	4	Loads the controller button bitmask into <code>D0</code> — a shortcut for reading <code>\$7E800</code> directly. See Reading the Gamepad above.
#6	<code>TRAP #6</code>	4	Writes <code>D0</code> (frequency), <code>D1</code> (duration), <code>D2</code> (volume), <code>D3</code> (waveform) into the sound registers and sets the trigger byte.
#7	<code>TRAP #7</code>	4	Pauses the program for <code>D0</code> milliseconds (word) — not <code>D1</code> , unlike <code>TRAP #6</code> 's duration field. See Waiting with TRAP #7 below.

Vector	Syntax	Cycles	Description
#8	TRAP #8	4	Pops the next queued keyboard character into D0 (0 if none pending). See Reading the Keyboard with TRAP #8 below.

Addressing a Pixel for TRAP #2/#3

TRAP #2 (read pixel) and TRAP #3 (write pixel) take a framebuffer *byte address* in A0, not x / y coordinates, so compute it yourself first. The framebuffer starts at \$40000 (see the [Memory Map](#)), each row is 320 pixels, and each pixel takes 4 bytes, so the address of pixel (x, y) is:

```
address = $40000 + (y * 320 + x) * 4
```

For example, to write a white pixel at (50, 10) :

```
MOVE.L #10,D0          ; y
MULU.W #320,D0
ADD.L #50,D0           ; + x
ASL.L #2,D0            ; * 4 bytes/pixel
ADD.L #$40000,D0
MOVE.L D0,A0
MOVE.L $FFFFFF,D0      ; white
TRAP #3                ; write it
```

Printing Text with TRAP #1

A0 points at a *null-terminated* string - ASCII bytes followed by a 0 byte that marks the end (the same convention as C strings). D0 is the x pixel and D1 the y pixel of the first character's top-left corner, and D2 is the 32-bit RGBA text color.

Every character is an 8x8 pixel cell, so the next one starts 8 pixels to the right. A newline byte (0A) drops to the next line, back at x = 0. Text that would run past the right edge wraps to the next line by itself. Only the letter shapes are drawn; the pixels behind them are left as they were. Bytes outside 20 - 7E draw nothing.

There is no scrolling: drawing past the bottom of the screen (or at a negative position) stops the program with an error.

TRAP #2 / #3 take a framebuffer address, not `x / y`: see [Addressing a Pixel for TRAP #2/#3](#) above.

Waiting with TRAP #7

The real MC68000 has no delay instruction and no TRAP reserved for one - TRAP #n on real hardware is a bare software interrupt, and Motorola never assigned any of the 16 vectors a fixed meaning. Machines built around it got precise pauses either from a busy-wait loop hand-timed to the CPU's clock speed, or from a separate hardware timer chip. TRAP #7 is a deliberate addition specific to this emulator, not a real 68000 facility, to spare a program written here from either of those.

D0 (word) is the delay in milliseconds - up to 65,535 (about 65 seconds); call it again for longer pauses. Careful: that's D0, not D1 - easy to mix up with TRAP #6 (see the [sound registers](#)), whose duration field is D1 (D0 there is frequency). Leaving D0 at whatever it held from an earlier instruction, instead of loading the delay into it, silently waits for that leftover value instead of the one you meant:

```
MOVE.W #500,D0 ; 500 ms
TRAP #7
```

TRAP #7 itself still executes instantly and the CPU still moves on to the next instruction right away, exactly like every other TRAP - it doesn't sit "inside" the delay. What differs is *when* that next instruction actually runs: the debugger's three ways of running a program treat the pending delay differently:

- **Run** honors it for real: the program genuinely pauses for the requested duration before the next instruction executes.
- **Pause**, hit while a delay is in progress, freezes it along with everything else - resuming Run later continues counting down from wherever it was, not from the moment TRAP #7 was first reached.
- **Step** always resolves a pending delay instantly. Single-stepping

means *you* are setting the pace already, so a `TRAP #7` never blocks a Step click - the next click just moves straight on to the following instruction.

Reading the Keyboard with TRAP #8

Every key you type is captured by the emulator and queued (up to 16 characters - typing faster than your program reads them just means it catches up later; the 17th character typed before that happens is dropped rather than growing the queue further). `TRAP #8` pops the *oldest* queued character into `D0`, or loads `D0` with `0` if nothing is waiting - call it in a loop each frame and keep reading while `D0` is nonzero to drain everything that piled up. `D0` is written as a full 32-bit long, zero-extended (`$000000xx`, never leftover garbage in the upper three bytes from an earlier instruction) - a plain `CMPI.L #0,D0` or `CMPI.B #0,D0` both work to test "nothing pending".

Only two ranges of characters are recognized: visible ASCII (`$20 - $7E` - letters, digits, punctuation, space) and the accented Latin-1 characters (`$A0 - $FF` - `é`, `à`, `ñ`, `¿`, `¡`, and similar). Anything else you press - Enter, Backspace, arrow keys, function keys, modifier keys, or a character outside both ranges (the Euro sign `€` and the French `œ` ligature, for instance, fall outside Latin-1) - is silently ignored and never queued. This is deliberate: `TRAP #8` hands your program raw characters, nothing more. If you want a real line editor - reading a name, letting the player correct a typo with Backspace - that logic (what Backspace does, where the cursor is, when the line is "done") is entirely up to your program to build out of the characters `TRAP #8` gives it; the emulator doesn't impose one.

Unlike every other `TRAP` in this table, there's no plain-memory equivalent for `TRAP #8` — reading the queue has a side effect (it removes the character), and giving that to a raw address would mean the debugger's read-only Memory Inspector could silently consume keystrokes just by displaying that address. `TRAP #8` is the only way to read it.

Example — testing for a specific key, both cases: a source-code `'A' / 'a'` is just a number (its ASCII code, `65 / 97` - see [Source Layout](#)), so comparing `D0` against one is an ordinary `CMPI`:

```

TRAP    #8           ; D0 = next key (0 if none pending)
CMPI.B  #'A',D0      ; uppercase A?
BEQ     GOT_A
CMPI.B  #'a',D0      ; lowercase a?
BEQ     GOT_A

```

Every letter's uppercase and lowercase codes differ by exactly `$20` (bit 5) - `A` is `$41`, `a` is `$61` - so once you have several letters to fold together, clearing that bit first is shorter than a pair of `CMPI` s per letter:

```

TRAP    #8
ANDI.B  #$DF,D0      ; clear bit 5: force uppercase
CMPI.B  #'A',D0      ; only need to test the one case now
BEQ     GOT_A

```

`ANDI.B #$DF,D0` only touches the byte the letter lives in, exactly like the `CMPI.B` above - safe precisely because `D0` 's upper three bytes are always zero after `TRAP #8`, per the width note above.

Example — echoing the key you just read: `TRAP #1` (see [Printing Text with TRAP #1](#) above) needs `A0` to point at a string *in memory*, and a register holds a value, not an address, so there's no way to hand it `D0` directly. The standard fix is a small buffer you declare once (its address never changes) but fill at runtime (its *content* does) - here, one byte for the character plus the `0` terminator `TRAP #1` needs:

```

POLL:
    TRAP    #8                ; D0 = next key (0 = none)
    TST.B   D0
    BEQ     POLL              ; nothing typed yet - keep polling

    MOVE.B  D0,CHAR           ; store the character
    LEA     CHAR,A0           ; A0 = the string to print
    MOVEQ   #10,D0
    MOVEQ   #10,D1
    MOVE.L  #$FFFFFFFF,D2     ; white
    TRAP    #1

CHAR:     DS.B    1           ; the character, set at runtime
          DC.B    0           ; null terminator

```

Typing is captured everywhere in the app except while your cursor is in the code editor or another text field - no need to click the Screen panel first.

Exceptions

Different from a `TRAP #n` a program calls on purpose: an exception is something the CPU raises *on its own* when an instruction hits a fault it can't just set a flag for. Only genuinely reserved/invalid encodings and runtime faults raise one — never an instruction this emulator simply hasn't implemented yet, which would run fine on real hardware.

Vector	Address	Raised by	Description
Zero Divide	\$40	DIVU / DIVS with a zero divisor	Jumps to the handler address stored at \$40.

Vector	Address	Raised by	Description
Illegal Instruction	\$44	MOVE.B to an address register; BTST / CHK / CLR / NEG / NOT / TST / ADDI / SUBI / ANDI / ORI / EORI / CMPI targeting one; ILLEGAL	Jumps to the handler address stored at \$44 . Every cause but the last is a reserved/undefined encoding on real 68000 hardware, not a missing feature; ILLEGAL raises this same one deliberately — see System above.
CHK	\$48	CHK 's bounds check failing (Dn < 0 or Dn > the upper bound)	Jumps to the handler address stored at \$48 . See CHK above.
TRAPV	\$4C	TRAPV executed with V set	Jumps to the handler address stored at \$4C . See System above.

Your program installs a handler by writing its address into the vector *before* the fault can happen:

```

MOVEA.L #$40,A0          ; the Zero Divide vector
MOVE.L  #HANDLER,(A0)    ; install the handler
...
DIVU.W  D1,D0            ; D1 = 0 -> jumps to HANDLER

```

Real 68000 hardware pushes the status register and PC onto a *supervisor* stack on any exception; this emulator has no supervisor-mode/status register concept, so only PC is pushed — onto A7 , exactly like JSR . So a handler ends with RTS , not the real RTE , to resume right after the faulting instruction — see [above](#) for why RTE itself, and the rest of the real 68000's system/privileged group, aren't implemented here. If no handler was installed when the fault happens, the emulator throws a clear error instead of jumping to address \$0 the way real (misconfigured) hardware would.

Inspecting Memory

The Debugger panel ends with a **Memory** section: a read-only hex dump of 192 bytes, 8 per row. Each row shows its address, the 8 bytes in hexadecimal, then the same bytes as ASCII text (a dot for anything unprintable). It opens on `$2000`, where programs are loaded.

- Type an address (`2000` , `$2000` or `0x2000`) and press Enter to jump there, or use the ◀ / ▶ buttons to move one page.
- The shortcut buttons jump to PC (the next instruction), Stack (where `A7` points), Program, Screen (the framebuffer), Input and Sound (the memory-mapped registers).
- Bytes your program just wrote are highlighted in red, and the byte at PC has a yellow outline.
- "Follow writes" (on by default) moves the window to the last address written each time you Step or Pause, so you don't have to hunt for it. It stays still while the program runs, and turns itself off when you navigate by hand; tick it again to resume.

Editing a byte

Click a byte to edit it: it turns into a two-digit hexadecimal field, already selected. Type the new value and press Enter to store it; the field then opens on the next byte, so to type a run of values you just alternate value and Enter (`FF` , Enter, `FF` , Enter...). After the last byte of the 128 shown, editing simply stops. Escape (or clicking elsewhere) cancels, and a field that is not valid hexadecimal (shown with a red outline) is refused.

- Editing works while the program is paused or stepping, and is switched off while it runs: the program would be overwriting memory thousands of times per frame, and your value would be lost or would corrupt whatever the program was using.
- A byte you edit by hand is not highlighted as a write by the program, and "Follow writes" does not jump to it.
- Editing a byte of the screen area repaints the screen immediately, but a single byte is hard to see: a pixel is 4 bytes (red, green, blue, alpha, in that order, starting at `$40000`) and the screen has 64,000 of them, so one byte changes one colour component of one pixel. The alpha byte

has no visible effect (the screen has no transparency). To see something, set all four bytes of a pixel: click its first byte, then type `FF`, Enter, `FF`, Enter, `FF`, Enter, `00`, Enter. That makes one white pixel (red, green and blue at full, alpha ignored). Each row of the dump holds two pixels.

- Nothing is protected: you can edit the bytes of the program itself, which patches it on the fly. The source shown in the editor then no longer matches what executes.
- Edits are lost when the program is loaded again: with Reset, or with the first Step or Run after a change to the source. An edit made before that first Step or Run is therefore wiped as soon as the program loads. Edit once the program has started (after one Step, for instance).
- Bytes only: to change a 16-bit or 32-bit value, edit each of its bytes. The 68000 stores the most significant byte first (big-endian), so `$1234` is the bytes `12` then `34`.

Writing Assembly Source

The assembler turns a text file of instructions into the machine code the CPU runs. The Debugger runs it for you: when you press Run or Step, your source is assembled (again only if it changed) and loaded into memory.

A tiny program:

```
        ORG      $2000          ; where the program is placed
START:                          ; a label: a name for this address
        MOVE.L   #100,D0        ; D0 = 100
        ADD.L    #5,D0          ; D0 = 105
        TRAP     #0             ; exit
        END      START          ; run from START
```

Source Layout

- A **label** names an address, so that a branch can say "go to LOOP" instead of counting bytes. It is a word starting in column 0, or any word ending in `:`.
- A **directive** is a command to the assembler itself (such as `ORG`), not a CPU instruction.

- Comments start with `;` (or `*` in column 0).
- Mnemonics, registers and directives are case-insensitive; labels are case-sensitive.
- Numbers: `123` , `$FF` (hex), `%1010` (binary), `'A'` (character code).
- Expressions combine numbers and labels with `+` and `-` only.
- Operands: `D0 - D7` , `A0 - A7` (`SP` means `A7`), `(A0)` , `(A0)+` , `-(A0)` , `4(A0)` , `#5` , and a bare address or label. `MOVEM` also takes a register list such as `D0-D2/A0` .
- A bare address or label is always stored as a 4-byte address.
- An instruction must sit at an even address (the 68000 reads instructions two bytes at a time); put `EVEN` before it after odd-sized data.

For example, the text `"Hi",0` is 3 bytes, so whatever follows lands on an odd address. `EVEN` adds one zero byte of padding to fix that:

```
MSG:    DC.B    "Hi",0        ; 3 bytes: next address is odd
        EVEN                               ; adds 1 padding byte: now even
START:  MOVE.L  #1,D0         ; OK: even address
```

Without the `EVEN` line, the assembler stops with "Instruction at an odd address".

Editor Shortcuts

Keys	Effect
<code>Tab</code>	Inserts spaces up to the next column that's a multiple of 8 (replaces the selection, if any) — lines up labels/mnemonics/operands the same way the examples do.
<code>Shift+Tab</code>	Removes up to 8 leading spaces from the current line.
<code>Enter</code>	Starts the new line with the same leading spaces as the current one.

Keys	Effect
<code>Ctrl+Z</code> (<code>Cmd+Z</code> on macOS)	Undo.
<code>Ctrl+Y</code> or <code>Ctrl+Shift+Z</code> (<code>Cmd+Shift+Z</code> on macOS)	Redo.
<code>Ctrl+C</code> / <code>Ctrl+X</code> (<code>Cmd+C</code> / <code>Cmd+X</code> on macOS)	Copy / cut. With a selection, the usual behavior. With no selection, copies/cuts the whole current line (see below).
<code>Ctrl+V</code> (<code>Cmd+V</code> on macOS)	Paste.
<code>Ctrl+O</code> (<code>Cmd+O</code> on macOS)	Open a file (same as the toolbar's Open...).
<code>Ctrl+S</code> (<code>Cmd+S</code> on macOS)	Save to the current file. Disabled (button greyed out, shortcut does nothing) when there isn't one — an example or the built-in starter program was never saved anywhere, so only Save As... is available until you pick a location.
<code>Ctrl+Shift+S</code> (<code>Cmd+Shift+S</code> on macOS)	Save As... — always asks for a location.
Right-click a line number, or <code>Ctrl+B</code>	Toggle a bookmark.
<code>F2</code> / <code>Shift+F2</code>	Jump to the next / previous bookmark.
<code>Ctrl+F</code> (<code>Cmd+F</code> on macOS)	Open the find bar.
<code>F3</code> / <code>Shift+F3</code> , or Enter in the find field	Jump to the next / previous match, wrapping around.

Keys	Effect
Escape	Close the find bar.

Undo/redo is the app's own history, not the browser's native text field undo (the desktop build doesn't deliver `Ctrl+Z` / `Ctrl+Y` to that one at all) — **Undo/Redo** buttons in the toolbar do the same thing for a mouse, greyed out whenever there's nothing to undo/redo. It only covers edits since the source was last loaded — opening a file or picking an example both start a fresh history. The Debugger's Reset button only resets the CPU and memory, not the source, so it doesn't affect undo/redo.

Opening a file or loading an example discards the current buffer, so both ask for confirmation first if it has unsaved changes — including when you've edited and then undone your way back to exactly the original content, since that discards the redo history too.

Copying or cutting with no text selected acts on the whole current line instead of doing nothing: the line (trailing spaces removed) plus a newline, so pasting it elsewhere inserts a full line, the same way most code editors handle it. **Copy/Cut/Paste** are also available as toolbar buttons (and in the commands menu) for the mouse — useful since cutting or pasting that way still respects the no-selection whole-line behavior for Copy/Cut.

The editor panel's title shows the file currently being edited (its name only, not the full path), or "Untitled" for an example or the built-in starter program that hasn't been saved anywhere yet; a `•` next to it means there are unsaved changes.

Directives

Directive	Effect
ORG addr	Places the following bytes at addr (default \$2000).
END [label]	Ends the source; the label is where execution starts.
DC.B/W/L a, b	Stores constants; DC.B also accepts "text".
DS.B/W/L n	Reserves n zeroed bytes, words or long words.

Directive	Effect
<code>EQU expr</code>	Makes the line's label a constant, not an address.
<code>EVEN</code>	Adds one zero byte if the address is odd.

Branch Sizes

A **branch** is an instruction that makes the CPU continue somewhere other than the next instruction: `BRA` (always), `Bcc` (only if a condition holds, e.g. `BNE`, `BEQ`), `BSR` (like `JSR`: remember where to come back to), and `DBcc` (a loop counter). You give it a **label** as the target, and the assembler works out the rest.

What the assembler actually stores in the instruction is not the label's address but a **distance**: how many bytes forward (positive) or backward (negative) the target is, measured from the address just after the branch's first word. Storing a distance, rather than an address, is what lets the same code run wherever it is loaded. The instruction has room for that distance in one of two sizes, which you can pick with a suffix on the mnemonic:

Suffix	Distance stored in	Reaches	Instruction size
<code>.s</code> (short)	8 bits	-128 to +127 bytes, but not 0	2 bytes
<code>.w</code> (word, the default)	16 bits	-32768 to +32767 bytes	4 bytes

```

LOOP:  SUBQ.L  #1,D0
        BNE.S  LOOP      ; close target: short form is enough
        BRA    FAR_AWAY  ; no suffix = .w, reaches much farther

```

Rules worth knowing:

- **No suffix means `.w`**, which is always the safe choice; `.s` only saves two bytes.

- **Too far is an error**, not a silent mistake: `BNE.S` to a label 200 bytes away reports a "displacement out of range" error. Change it to `BNE.W` (or drop the suffix). The assembler never switches sizes on your behalf.
- **.S cannot branch to the very next instruction** (a distance of 0 would be read as "a 16-bit distance follows"); use `.W` or just remove the branch.
- **DBcc has no suffix**: its distance is always 16 bits.
- **JMP and JSR are not branches** in this sense: they take a full address, so distance never limits them.

Assemblable Instructions

Every real mnemonic is assemblable. `CCR` and `SR` are accepted as operands only where they make sense: `MOVE <ea>,CCR`, `MOVE SR,<ea>` and `ANDI / ORI / EORI #imm,CCR`. `MOVE <ea>,SR` and the other privileged instructions are not part of this emulator.

Category	Mnemonics
Data Movement	<code>MOVE</code> , <code>MOVEA</code> , <code>MOVEQ</code> , <code>MOVEM</code> , <code>MOVEP</code> , <code>LEA</code> , <code>PEA</code> , <code>EXG</code> , <code>SWAP</code> , <code>EXT</code>
Arithmetic	<code>ADD</code> , <code>SUB</code> , <code>CMP</code> , <code>ADDA</code> , <code>SUBA</code> , <code>CMPA</code> , <code>ADDI</code> , <code>SUBI</code> , <code>CMPI</code> , <code>ADDQ</code> , <code>SUBQ</code> , <code>ADDX</code> , <code>SUBX</code> , <code>CMPM</code> , <code>CLR</code> , <code>TST</code> , <code>NEG</code> , <code>NEGX</code> , <code>MULU</code> , <code>MULS</code> , <code>DIVU</code> , <code>DIVS</code> , <code>ABCD</code> , <code>SB CD</code> , <code>NBCD</code>
Logical	<code>AND</code> , <code>OR</code> , <code>EOR</code> , <code>ANDI</code> , <code>ORI</code> , <code>EORI</code> (also <code>#imm,CCR</code>), <code>NOT</code> , <code>ScC</code>
Bit Manipulation	<code>BTST</code> , <code>BCHG</code> , <code>BCLR</code> , <code>BSET</code> , <code>TAS</code>
Shift and Rotate	<code>ASL</code> , <code>ASR</code> , <code>LSL</code> , <code>LSR</code> , <code>ROL</code> , <code>ROR</code> , <code>ROXL</code> , <code>ROXR</code>
Branches	<code>BRA</code> , <code>BSR</code> , <code>Bcc</code> (all conditions), <code>DBcc</code> , <code>DBRA</code>

Category	Mnemonics
Subroutine Control	<code>JMP</code> , <code>JSR</code> , <code>RTS</code> , <code>RTR</code> , <code>LINK</code> , <code>UNLK</code>
System	<code>NOP</code> , <code>TRAP</code> , <code>TRAPV</code> , <code>CHK</code> , <code>ILLEGAL</code>

Assembler Limits

- A label defined by `EQU` must appear before it is used, and `ORG` and `DS` counts cannot use labels defined later.
- `DC` values must fit their width (`DC.B` accepts -128..255) and `DC.W` / `DC.L` / `DS.W` / `DS.L` must sit at an even address (put `EVEN` before them); a size suffix the mnemonic cannot take (`NOP.L` , `MULU.L`) is an error.
- `d(PC)` and `d(PC,Xn)` take the target label as `d` (e.g. `LEA table(PC),A0`); the assembler works out the distance. They are source-only, and `BTST #n,d(PC)` is not assemblable.
- Errors are reported all at once as a list of line, column and message.

Assembly Programming Tips

Declaring data: `DC.B`, `DC.W`, `DC.L`

On real 68000 assemblers, `DC.B` , `DC.W` , and `DC.L` ("Define Constant") reserve space in memory and fill it with fixed values at assembly time — the usual way to embed a lookup table, a string, or any other static data alongside the code, e.g.:

```

ARRAY:
    DC.L    10, 20, 30, 40, 50    ; five longs, back to back
MESSAGE:
    DC.B    "HI",0                ; bytes: 'H', 'I', 0

```

The assembler accepts `DC.x` (see [Writing Assembly Source](#)). An alternative is building the data at *runtime*: pick an address, then write each value into it with its own `MOVE` , stepping the address with `(An)+` the

same way a program would read the data back out later. The memory ends up holding exactly the same bytes.

Reserving space: DS.B, DS.W, DS.L

Where `DC.x` fills memory with values known at assembly time, `DS.x` ("Define Storage") does the opposite: it reserves `n` zero-filled bytes, words or long words without giving them a starting value — the usual way to set aside a buffer or a scratch variable that a running program will write into later, e.g.:

```
BUFFER: DS.B    10                ; 10 zeroed bytes, nothing set yet
COUNT: DS.W    1                ; one zeroed word
```

Same caveats as `DC.W / DC.L`: `DS.W` and `DS.L` must start at an even address (`EVEN` before them if needed — see the `FLAG` example under [Directives](#)). `n` doesn't have to be a plain number — a label or an `EQU` constant works too, as long as it's already defined earlier in the source:

```
LEN:      EQU    20
TABLE:    DS.B    LEN                ; ok: LEN is defined above
```

Swap the two lines and it fails to assemble — at that point in the source, `LEN` hasn't been defined yet (see [Assembler Limits](#)).

Naming constants: EQU

`DC / DS` both reserve a byte, word or long word *in memory*, at an address the label refers to. `EQU` is different: it never touches memory, it just gives a name to a fixed value the assembler substitutes wherever that name appears, e.g.:

```
MAXLEN: EQU    10                ; not an address, just 10
BUFFER: DS.B    MAXLEN+1         ; reads as "DS.B 11"
```

Since `MAXLEN` never becomes a real address, you can't use `MOVE` on it or index through it with `(An)` the way you would with a `DC / DS` label — `MAXLEN` in code is replaced by the literal `10`, nothing more. This is what makes it useful for a size or a repeated magic number: change the one

`EQU` line and every use of the name picks it up. There are no forward references: an `EQU` line must come before every place that uses its name (see [Assembler Limits](#)).

Writing the same number four ways

An immediate operand (`#...`) or a `DS` count doesn't have to be a plain decimal number — the assembler also accepts hexadecimal, binary, a character's ASCII code, and an `EQU` name. These four lines all load the exact same value:

```
MOVE.B  #65,D0           ; decimal
MOVE.B  #$41,D0          ; hexadecimal
MOVE.B  #%01000001,D0    ; binary
MOVE.B  #'A',D0          ; character code
```

Pick whichever reads clearest for what the number *means* — `$41` for a raw byte value, `%01000001` for a bit pattern you're building up, `'A'` when the number is really a letter (see [Reading the Keyboard with TRAP #8](#) for a worked example). An `EQU` name works the same way once it's declared, as `#LEN` would above once `LEN: EQU 65` exists earlier in the source.

Tips

A few habits worth having, some of them straight from gotchas this emulator's own opcodes hit during development:

- **Initialize every register before reading it.** Nothing clears `D0 - A7` to a known value automatically — a register holds whatever was last written to it (they do start at `0`, but don't rely on that once a program is running).
- **CMP first, Bcc second, and pick the right family.** Mixing up signed and unsigned comparisons is the classic bug — see [Which Bcc do I want?](#) if a branch seems to go the wrong way.
- **DBcc / DBRA loop counters start at N - 1, not N.** The loop body always runs once before the first decrement — see [How does DBcc decide?](#) for the full walk-through; it's an easy off-by-one to trip over.

- **Save what a subroutine will clobber, restore it before returning.**
`MOVEM` pushing onto `-(A7)` at the top and popping from `(A7)+` at the bottom is the standard pattern — see its worked example above.
- **`MOVEA` / `ADDQ` / `SUBQ` to an address register never touch the flags,** and are always a full 32-bit operation regardless of the size field — real 68000 behavior, easy to forget if a flag check right after one comes back stale.
- **Prefer `MOVEQ` for small constants (`-128` to `127`)** into a data register — it's the same 4 cycles as `MOVE.L #imm, Dn` but a shorter encoding, and shifts (`ASL` / `LSL` , etc.) are a cheap way to multiply or divide by a power of two instead of reaching for `MULU` / `DIVU` .
- **Comment the *why*, not just the *what*, even in illustrative examples.**
 Every worked example on this page does — it's what makes a hand-traced instruction sequence checkable later.

Running a Program

The interface is available in English, French and Spanish. It starts in your browser's or system's language (English if that isn't one of the three); change it with the **Language** list under the gamepad, and the choice is remembered. This guide names the controls by their English labels. The toolbar above the registers has four controls:

- **Run / Pause** (*Exécuter* / *Ejecutar* in French / Spanish) starts or stops continuous execution. Run also stops by itself on a breakpoint (see [My breakpoint is ignored](#)), a runtime error, or the program's exit.
- **Step** (*Pas à pas* / *Paso*) executes exactly one instruction, ignores breakpoints, and is only available while the program is paused.
- **Reset** (*Réinitialiser* / *Reiniciar*) puts the CPU back to its initial state.
- **Speed** (*Vitesse* in French, *Velocidad* in Spanish) is the drop-down list (`10` , `200` , `2000` or `20000` instructions per refresh). It sets how many instructions Run executes per *refresh*. A refresh is one redraw of the screen: the emulated screen is repainted about 60 times per second on a typical display, and between two repaints the CPU runs the chosen number of instructions. Hovering the list, or clicking the ? button between the label and the list (the way to do it on a touch screen), shows this explanation.

At the default `2000 instructions per refresh`, that is about 120,000 instructions per second. `10` gives about 600 per second, slow enough to follow a program by eye, and `20000` about 1.2 million per second, for long computations. Speed only changes how fast the program runs on screen, never what it computes, and it has no effect on Step.

The screen is repainted once per refresh, so a low Speed also makes a drawing program appear pixel by pixel. Speed does not change the cycle counter, which counts the emulated CPU's instructions, not real time (see [Performance Notes](#)).

Performance Notes

- **Simple interpretation, no recompilation:** each instruction is fetched, decoded, and executed one at a time — there's no JIT, no bytecode caching. That keeps the implementation easy to follow, which matters more here than raw speed for hand-written assembly programs at this scale.
- **Cycle costs are currently flat per instruction** (see the tables above), not the real 68000's addressing-mode-dependent timing — today's cycle counter is a rough guide for comparing programs, not a cycle-accurate simulation of real hardware.
- **Memory access is O(1)** everywhere: no caching, no virtual memory — the whole address space (RAM, framebuffer, controller input) is backed by one flat block of memory, so reading or writing any address costs the same.
- **No memory protection *inside* the emulated address space:** a running program can read or write any address there, including the system area — nothing stops a bug from overwriting its own vector table or code. This matches real 68000 hardware, which has no MMU either (the same was true of the Mac 128K, Sega Genesis, and Atari ST). It does **not** mean a buggy program can touch your actual computer's memory — the whole emulated space is one self-contained block inside the app; there's no bridge to your real machine, so there's nothing to "escape" to. Out-of-bounds addresses (outside `$000000` – `$7E803`) are still rejected and stop the program.

Troubleshooting

Answers for the things most likely to trip you up while working through the [Reference](#) today — like the rest of this guide, this page only covers what's real right now, not the final wish list (see [Presentation](#) for what's still ahead).

The Program Won't Run Properly

The editor, Run button or debugger doesn't behave as expected: a breakpoint is ignored, the highlighted line seems stuck, the program never stops, or it runs too slowly.

"My breakpoint is ignored"

A breakpoint is a red line number in the editor's left margin (click a number to set or remove it). **Run** stops when the *next instruction to execute* sits on a marked line — the line highlighted in yellow. A breakpoint silently does nothing in these cases:

- **The line holds no instruction.** Blank lines, comments, a label alone on its line, and directives (`ORG` , `EQU` , `END` , `EVEN`) produce no code, so the CPU never arrives there. Put the breakpoint on the instruction line itself. Data lines (`DC` , `DS`) are only "reached" if the CPU actually executes them as code, which a working program doesn't.
- **The instruction is never reached.** It sits after a `TRAP #0` , in a branch that is never taken, or in a subroutine that is never called.
- **It's on the very first instruction.** The check happens *after* each instruction runs, so pressing Run from the start executes the first instruction before looking at breakpoints. Use **Step** for that one. The same applies when you resume from a breakpoint: the instruction you're stopped on runs first (a loop that comes back to it will stop there again).
- **You're using Step.** Step always executes exactly one instruction and ignores breakpoints.

- **You edited its line structure.** A breakpoint follows its instruction when you insert or delete lines around it, and survives retyping its own line. It is removed if its line is deleted, split in two with Enter, or replaced together with neighbouring lines (a pasted block, whatever its size) — set it again afterwards.

"My program never finishes / seems to hang"

Cause: an infinite loop — most often a `BRA` that branches back to itself (or to a point before it) with no way out:

```
LOOP:
; ... code ...
BRA    LOOP    ; loops forever – nothing ever branches away
```

A `DBcc / DBRA` loop can do the same thing if its counter never reaches its stop condition — see [How does DBcc decide?](#).

What to check today: **Pause** stops a running program, and a breakpoint inside the loop (see [My breakpoint is ignored](#)) lets you watch its registers change. Hand-tracing the loop's exit condition — the way every worked example on the Reference page does — is the reliable way to catch this before it becomes a real problem.

"The highlighted line doesn't follow my program while it runs"

While **Run** is going, the yellow line is refreshed only once per screen refresh, after that many instructions (see **Speed**). A program that spends most of its time in a waiting loop — like the `WAIT: DBRA D4, WAIT` loop of example 13, "Playing a Scale" (editor's "Load an example" menu) — is nearly always caught inside that loop, so a line executed once per note, like `TRAP #6`, is never the one shown. The CPU does run every line; the display just doesn't sample it.

To watch the flow:

- Use **Step**, which advances exactly one instruction.
- Put a breakpoint on the line you want to see (see [My breakpoint is ignored](#)): **Run** stops there.

- Lower the **Speed** setting, at the price of a slower program (a waiting loop lasts longer too).

"Program runs too slowly"

The emulator's cycle counter is currently a flat number per instruction, not the real 68000's addressing-mode-dependent timing — see [Performance Notes](#). If a program's cycle count looks unexpectedly high:

- Raise the **Speed** setting above the registers (up to 20000 instructions per refresh) — see [Running a Program](#).
- Millions of loop iterations add up on their own — the counter isn't wrong, a large loop bound might just be the actual cause.
- Prefer `MOVEQ` over `MOVE.L #imm` for small constants, and shifts (`ASL` / `LSL`, etc.) over `MULU` / `DIVU` for power-of-two math — see [Tips](#).
- Minimize memory access inside a hot loop where a register would do just as well — each cycle interprets exactly one instruction, with no recompilation to optimize a hot path away (see [Performance Notes](#)).

The Program Runs but Gives Wrong Results

The program runs to the end, but registers or values end up different from what you expected.

"Registers don't change the way I expect"

Check the size suffix first — it decides how much of the register actually gets written:

```
MOVE.W #100,D0    ; only the low word changes
MOVE.L #100,D0    ; all 32 bits change
```

And check the addressing mode — `#value` loads a constant, a bare number loads *from* that address instead:

```
MOVE.L #100,D0    ; correct: D0 = 100
MOVE.L 100,D0     ; different instruction: D0 = Memory[100]
```

"A subroutine call corrupts registers it shouldn't touch"

`JSR / RTS` only manage the return address — nothing about a subroutine call saves any register automatically. If a subroutine needs `D0 - D7 / A0 - A6` as scratch without disturbing the caller's values, it has to save and restore them itself, typically `MOVEM` pushing onto `-(A7)` on the way in and popping from `(A7)+` on the way out — see [How does MOVEM's register list work?](#) for the worked example.

"A calculation gives the wrong result"

The usual causes:

- **Signed vs. unsigned confusion** in a comparison — the same bit pattern branches differently depending on which `Bcc` family reads it. See [Which Bcc do I want?](#).
- **Overflow** — check `V` (and `C` for unsigned) after the operation; both `ADD / SUB` and their `Q`-immediate forms set them.
- **An off-by-one in a `DBcc / DBRA` loop count** — see [How does DBcc decide?](#).

The Gamepad Doesn't Behave

"My physical controller's buttons are wrong or the D-pad gets stuck"

Only the D-pad, A/B/X/Y, Start and Select are used; the shoulder buttons, triggers, stick clicks and the center (Home) button are ignored. The left stick also works as a D-pad.

- **Some controllers have a mode switch** (for example X / O, or Xbox / PlayStation). If the four face buttons don't light up the matching on-screen buttons, try the other position.
- **On a Nintendo-style controller, A/B and X/Y look swapped.** They are not a bug: the four face buttons follow their position (A bottom, B right, X left, Y top), as on an Xbox controller, so the buttons printed "A" and "B" (and "X" and "Y") trade places. See [Reading the Gamepad](#).

- **Windows: the controller isn't detected at all.** The app only accepts controllers that Windows presents as a standard (Xbox-style) gamepad. Put the controller in the mode that presents it as an Xbox controller (on the Nacon GC-100, the **X** position) and it works; in the other mode Windows reports it as a generic device and the app ignores it.
- **Stuck directions after pressing the triggers** (Linux, versions up to 0.4.2): the D-pad stopped responding and *up* stayed held, and on some controllers a face button lit the wrong on-screen button. Fixed in later versions; if you still see it, update the app.
- **Linux only:** to see what the native controller layer receives, start the app from a terminal with `RETRO68K_GAMEPAD_DEBUG=1` set in front of the command; every controller event is printed there. Include that output when [reporting an issue](#).

Getting More Help

- **Compare with the built-in examples:** fifteen complete, working programs (loops, framebuffer writes, subroutines, gamepad...) to check your code against, in the editor's "Load an example" menu.
- **Re-read the relevant [Reference row](#)** for the instruction in question — its Description and Flags columns call out the gotchas already known.
- **Hand-trace the instructions**, the way every worked example on the Reference page does — writing out each register's value line by line is usually where a wrong assumption becomes visible.
- **Simplify:** cut the program down to the smallest sequence that still shows the problem.

Reporting Issues

If something here doesn't explain what you're seeing, or you've found a real bug, include when you report it:

1. The instruction sequence involved (hand-traced, if you can)
2. What you expected vs. what actually happened
3. The app version (or commit, if running from source)

4. Steps to reproduce

Back to [Presentation](#) · [Downloads](#) · [Reference](#)

Retro 68K Emulator — User Guide

Copyright 2026 — free software